

CNN Stress Detection + Quantization Comparison Alur Notebook: A. Data Preparation → B. Dataset Restructuring → C. Preprocessing → D. Training → E. Evaluation → F. Quantization → G. Comparison → H. Conclusion

```
from google.colab import files
uploaded_fer = files.upload()
```

Choose Files

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving clean_dataset.zip to clean_dataset.zip

A. DATA PREPARATION

A.1 IMPORT LIBRARY

```
# A.1 IMPORT LIBRARY

import os, time, copy, shutil, zipfile
import torch
import torch.nn as nn
import torch.optim as optim
import torch.quantization
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
from tqdm import tqdm
import pandas as pd
from sklearn.metrics import f1_score, confusion_matrix, classification_report
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Device:', device)

# RANDOM SEED (REPRODUCIBILITY)

import random

SEED = 42

random.seed(SEED)
np.random.seed(SEED)

torch.manual_seed(SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed(SEED)
    torch.cuda.manual_seed_all(SEED)

torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

print(f"Random seed set to {SEED}")
```

Device: cuda
Random seed set to 42

A.2 UPLOAD & EXTRACK DATASETS

```
# A.2 UPLOAD & EXTRACT ZIP
zip_path = '/content/clean_dataset.zip'

# Cek file ZIP
if os.path.exists(zip_path):
    file_size = os.path.getsize(zip_path)
    print(f"File '{zip_path}' ditemukan. Ukuran: {file_size} bytes")
else:
    print(f"ERROR: File '{zip_path}' tidak ditemukan. Upload dulu ke Colab!")

# Ekstrak ZIP
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall('/content/clean_dataset')
print('Ekstraksi selesai!')
```

File '/content/clean_dataset.zip' ditemukan. Ukuran: 123487788 bytes
Ekstraksi selesai!

A.3 CEK STRUKTUR FOLDER DAN JUMLAH GAMBAR PER KELAS

```
# A.3 CEK STRUKTUR FOLDER

ORIGINAL_DIR = '/content/clean_dataset/images'

print('Struktur Folder & Jumlah Gambar')
for split in sorted(os.listdir(ORIGINAL_DIR)):
    split_path = os.path.join(ORIGINAL_DIR, split)
```

```
if not os.path.isdir(split_path):
    continue
for label in sorted(os.listdir(split_path)):
    label_path = os.path.join(split_path, label)
    if os.path.isdir(label_path):
        n = len(os.listdir(label_path))
        print(f' {split}/{label}: {n} gambar')
```

Struktur Folder & Jumlah Gambar

images/train:	7 gambar
images/validation:	7 gambar
train/angry:	3993 gambar
train/disgust:	436 gambar
train/fear:	4103 gambar
train/happy:	7164 gambar
train/neutral:	4982 gambar
train/sad:	4938 gambar
train/surprise:	3205 gambar
validation/angry:	960 gambar
validation/disgust:	111 gambar
validation/fear:	1018 gambar
validation/happy:	1825 gambar
validation/neutral:	1216 gambar
validation/sad:	1139 gambar
validation/surprise:	797 gambar

A.4 CEK & HAPUS DUPLIKAT TRAIN - VALIDATION (MENCEGAH DATA LEAKAGE)

```
# A.4 CEK DUPLIKAT TRAIN-VALIDATION
# Kumpulkan semua nama file di train
train_files = set()
train_base = os.path.join(ORIGINAL_DIR, 'train')
val_base   = os.path.join(ORIGINAL_DIR, 'validation')

for cls in os.listdir(train_base):
    cls_path = os.path.join(train_base, cls)
    if os.path.isdir(cls_path):
        for f in os.listdir(cls_path):
            train_files.add(f)

# Cek overlap di validation
overlap_count = 0
for cls in os.listdir(val_base):
    cls_path = os.path.join(val_base, cls)
    if not os.path.isdir(cls_path):
        continue
    for f in os.listdir(cls_path):
        if f in train_files:
            overlap_count += 1
            os.remove(os.path.join(cls_path, f)) # Hapus duplikat dari validation

print(f'Duplikat ditemukan dan dihapus dari validation: {overlap_count} file')
print('Data leakage sudah dicegah.')

# Cek ulang jumlah setelah pembersihan
print('\nJumlah Gambar Setelah Pembersihan Duplikat')
for split in ['train', 'validation']:
    split_path = os.path.join(ORIGINAL_DIR, split)
    if not os.path.isdir(split_path):
        continue
    for label in sorted(os.listdir(split_path)):
        label_path = os.path.join(split_path, label)
        if os.path.isdir(label_path):
            n = len(os.listdir(label_path))
            print(f' {split}/{label}: {n} gambar')
```

Duplikat ditemukan dan dihapus dari validation: 0 file
Data leakage sudah dicegah.

Jumlah Gambar Setelah Pembersihan Duplikat

train/angry:	3993 gambar
train/disgust:	436 gambar
train/fear:	4103 gambar
train/happy:	7164 gambar
train/neutral:	4982 gambar
train/sad:	4938 gambar
train/surprise:	3205 gambar
validation/angry:	960 gambar
validation/disgust:	111 gambar
validation/fear:	1018 gambar
validation/happy:	1825 gambar
validation/neutral:	1216 gambar
validation/sad:	1139 gambar
validation/surprise:	797 gambar

B. DATASET RESTRUCTURING Menggabungkan 7 kelas ekspresi wajah menjadi 2 kelas: stress dan non_stress.

```
# B. PEMBENTUKAN DATASET 2 KELAS
stress_classes    = ['angry', 'fear', 'sad', 'disgust']
non_stress_classes = ['happy', 'neutral', 'surprise']
splits = ['train', 'validation']
```

```
NEW_DIR = '/content/clean_dataset_2class'

# Bersihkan folder lama
if os.path.exists(NEW_DIR):
    shutil.rmtree(NEW_DIR)

# Buat folder baru
for split in splits:
    os.makedirs(os.path.join(NEW_DIR, split, 'stress'), exist_ok=True)
    os.makedirs(os.path.join(NEW_DIR, split, 'non_stress'), exist_ok=True)

# Copy file ke folder baru
for split in splits:
    for cls in stress_classes:
        src = os.path.join(ORIGINAL_DIR, split, cls)
        if os.path.exists(src):
            for f in os.listdir(src):
                shutil.copy(os.path.join(src, f),
                             os.path.join(NEW_DIR, split, 'stress', f))
    for cls in non_stress_classes:
        src = os.path.join(ORIGINAL_DIR, split, cls)
        if os.path.exists(src):
            for f in os.listdir(src):
                shutil.copy(os.path.join(src, f),
                             os.path.join(NEW_DIR, split, 'non_stress', f))

# Verifikasi hasil
print('Dataset 2 Kelas')
for split in splits:
    for cls in ['stress', 'non_stress']:
        p = os.path.join(NEW_DIR, split, cls)
        print(f' {split}/{cls}: {len(os.listdir(p))} gambar')
```

Dataset 2 Kelas
train/stress: 13470 gambar
train/non_stress: 15351 gambar
validation/stress: 3228 gambar
validation/non_stress: 3838 gambar

```
# VISUALISASI DISTRIBUSI DATASET 2 KELAS
splits = ["Train", "Validation"]
stress = [13470, 3228]
non_stress = [15351, 3838]

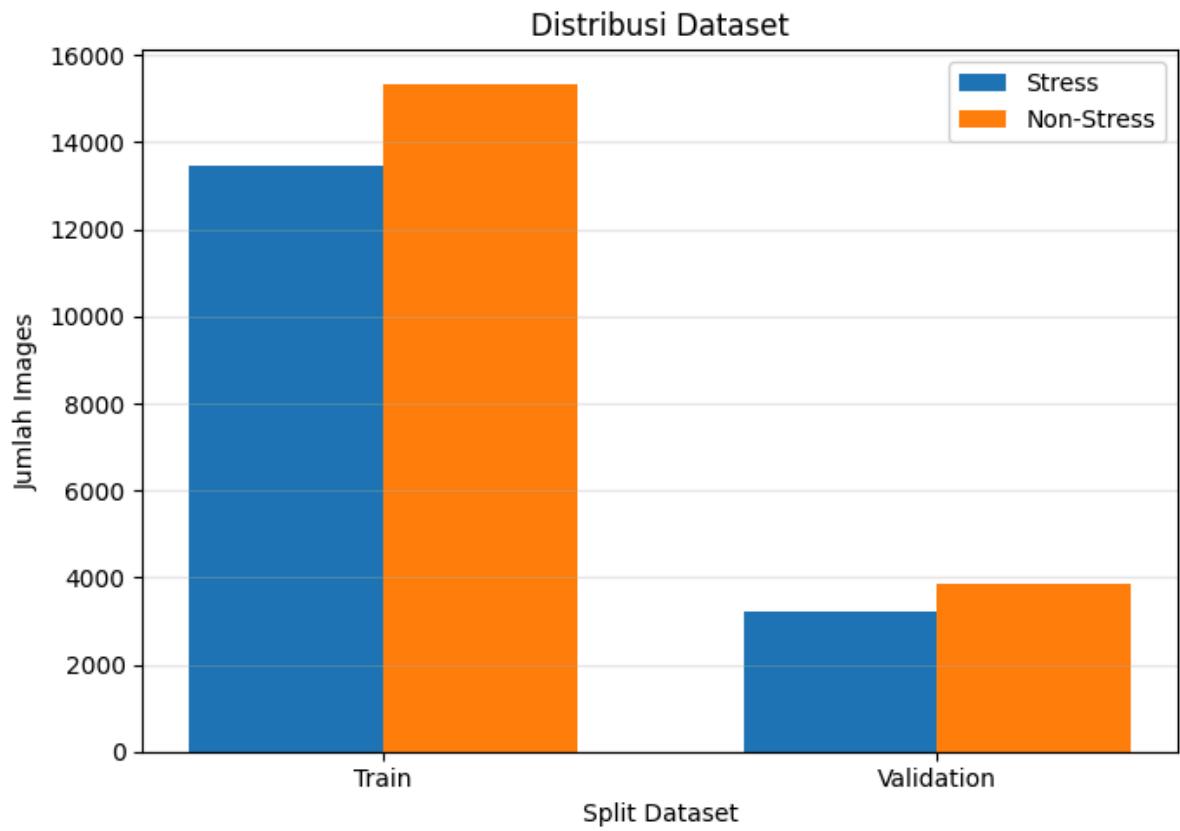
x = np.arange(len(splits))
width = 0.35

plt.figure(figsize=(7, 5))

plt.bar(x - width/2, stress, width, label="Stress")
plt.bar(x + width/2, non_stress, width, label="Non-Stress")

plt.title("Distribusi Dataset")
plt.xlabel("Split Dataset")
plt.ylabel("Jumlah Images")
plt.xticks(x, splits)
plt.legend()
plt.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.show()
```



C. PREPROCESSING & AUGMENTATION

```

# C. PREPROCESSING + AUGMENTATION
# Augmentasi hanya untuk TRAIN; validation hanya resize + normalize

train_tfms = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((48, 48)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1), scale=(0.9, 1.1)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5])
])

val_tfms = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((48, 48)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5])
])

DATA_DIR = NEW_DIR
train_dir = os.path.join(DATA_DIR, 'train')
val_dir = os.path.join(DATA_DIR, 'validation')

train_dataset = datasets.ImageFolder(train_dir, transform=train_tfms)
val_dataset = datasets.ImageFolder(val_dir, transform=val_tfms)

generator = torch.Generator()
generator.manual_seed(SEED)

train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True,
    num_workers=2,
    generator=generator
)

val_loader = DataLoader(
    val_dataset,
    batch_size=64,
    shuffle=False,
    num_workers=2,
    generator=generator
)

num_classes = len(train_dataset.classes)
print('Classes      : ', train_dataset.classes)
print('Num classes : ', num_classes)
print('Train samples:', len(train_dataset))
print('Val samples  : ', len(val_dataset))

```

```

Classes      : ['non_stress', 'stress']
Num classes : 2
Train samples: 28821
Val samples  : 7066

```

```

# VISUALISASI AUGMENTASI
images, labels = next(iter(train_loader))

plt.figure(figsize=(12,6))

for i in range(8):
    img = images[i].squeeze().numpy()
    img = (img * 0.5) + 0.5

    plt.subplot(2,4,i+1)
    plt.imshow(img, cmap='gray')
    plt.axis('off')
    plt.title(train_dataset.classes[labels[i]])

plt.suptitle("Sample Training Images After Augmentation")
plt.tight_layout()
plt.show()

```


Sample Training Images After Augmentation



```
# VISUALISASI DATA AUGMENTATION

from PIL import Image

# Ambil 1 gambar asli dari dataset train
sample_path = train_dataset.samples[0][0]
sample_label = train_dataset.samples[0][1]

# Baca gambar asli
original_image = Image.open(sample_path).convert('L')

# Augmentasi khusus training
augment_transform = transforms.Compose([
    transforms.Resize((48, 48)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15),
    transforms.RandomAffine(
        degrees=0,
        translate=(0.1, 0.1),
        scale=(0.9, 1.1)
    ),
    transforms.ToTensor()
])

fig, axes = plt.subplots(2, 5, figsize=(15, 6))

fig.suptitle(
    f'Data Augmentation - "{train_dataset.classes[sample_label]}"',
    fontsize=16,
    fontweight='bold'
)

# Gambar asli
axes[0, 0].imshow(original_image, cmap='gray')

axes[0, 0].set_title(
    'Original',
    fontsize=11,
    fontweight='bold',
    color='blue'
)

axes[0, 0].axis('off')

# 9 hasil augmentasi
for i in range(1, 10):

    row = i // 5
    col = i % 5

    augmented = augment_transform(original_image)

    axes[row, col].imshow(
        augmented.squeeze(),
        cmap='gray'
    )

    axes[row, col].set_title(
```

```

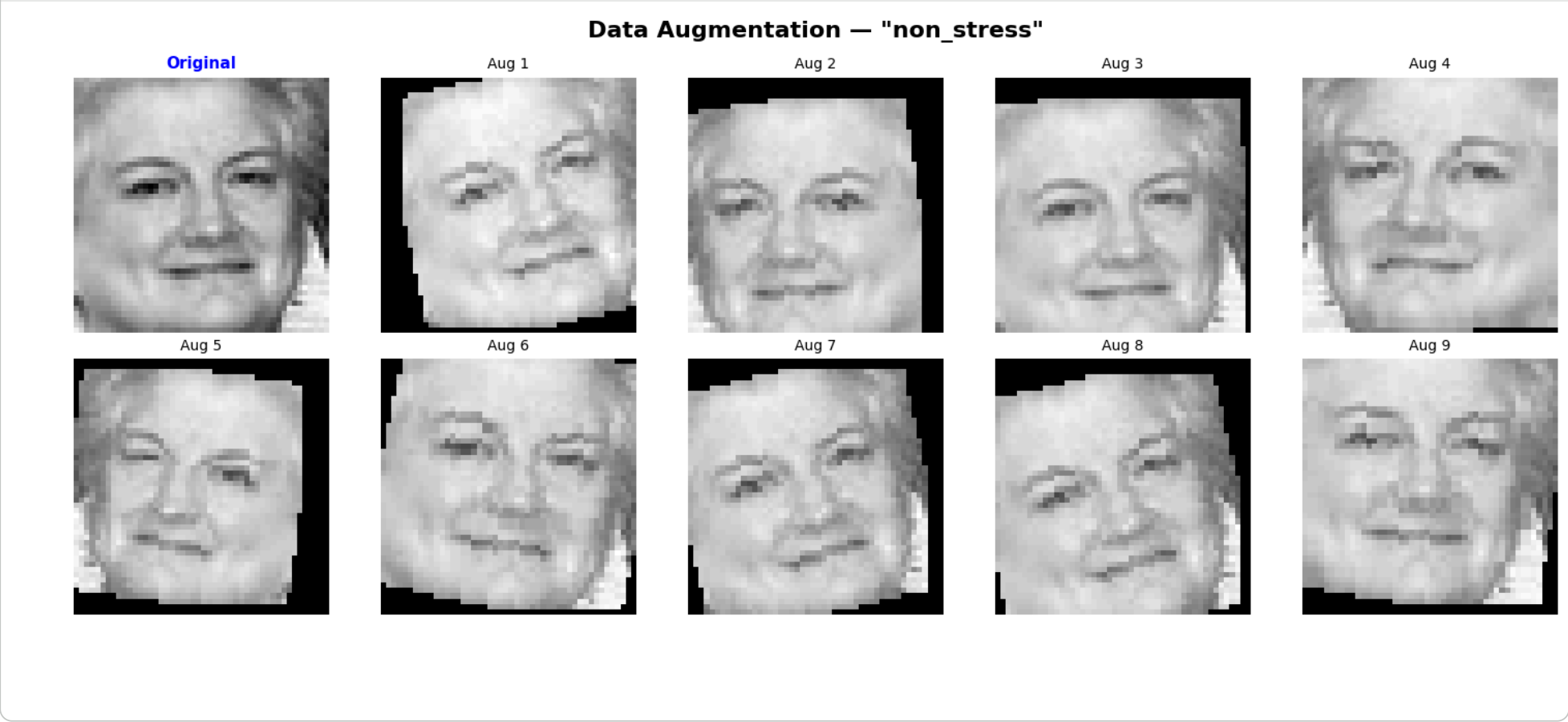
        f'Aug {i}',
        fontsize=10
    )

    axes[row, col].axis('off')

plt.tight_layout()

plt.show()

```



D. TRAINING MODEL CNN

```

# D.1 DEFINISI MODEL CNN
class EmotionCNNDeepFC(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()
        self.features = nn.Sequential(
            # Block 1
            nn.Conv2d(1, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2), nn.Dropout2d(0.25),
            # Block 2
            nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.Conv2d(128, 128, kernel_size=3, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.MaxPool2d(2), nn.Dropout2d(0.25),
            # Block 3
            nn.Conv2d(128, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256), nn.ReLU(),
            nn.Conv2d(256, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256), nn.ReLU(),
            nn.MaxPool2d(2), nn.Dropout2d(0.30),
        )
        # input 48x48 → setelah 3x MaxPool2d → 6x6
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(256 * 6 * 6, 1024), nn.BatchNorm1d(1024), nn.ReLU(), nn.Dropout(0.50),
            nn.Linear(1024, 512), nn.BatchNorm1d(512), nn.ReLU(), nn.Dropout(0.40),
            nn.Linear(512, 256), nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.30),
            nn.Linear(256, 128), nn.ReLU(), nn.Dropout(0.20),
            nn.Linear(128, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

model = EmotionCNNDeepFC(num_classes=num_classes).to(device)
print(model)

```

```

EmotionCNNDeepFC(
  (features): Sequential(
    (0): Conv2d(1, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (5): ReLU()
    (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (7): Dropout2d(p=0.25, inplace=False)
    (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): ReLU()
    (11): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (12): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (13): ReLU()

```

```

(14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
(15): Dropout2d(p=0.25, inplace=False)
(16): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(17): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(18): ReLU()
(19): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(20): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
(21): ReLU()
(22): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
(23): Dropout2d(p=0.3, inplace=False)
)
(classifier): Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=9216, out_features=1024, bias=True)
  (2): BatchNorm1d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (3): ReLU()
  (4): Dropout(p=0.5, inplace=False)
  (5): Linear(in_features=1024, out_features=512, bias=True)
  (6): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (7): ReLU()
  (8): Dropout(p=0.4, inplace=False)
  (9): Linear(in_features=512, out_features=256, bias=True)
  (10): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (11): ReLU()
  (12): Dropout(p=0.3, inplace=False)
  (13): Linear(in_features=256, out_features=128, bias=True)
  (14): ReLU()
  (15): Dropout(p=0.2, inplace=False)
  (16): Linear(in_features=128, out_features=2, bias=True)
)
)

```

```
from torchsummary import summary
```

```
print("\nMODEL SUMMARY")
summary(model, (1, 48, 48))
```

MODEL SUMMARY

Layer (type)	Output Shape	Param #
=====		
Conv2d-1	[-1, 64, 48, 48]	640
BatchNorm2d-2	[-1, 64, 48, 48]	128
ReLU-3	[-1, 64, 48, 48]	0
Conv2d-4	[-1, 64, 48, 48]	36,928
BatchNorm2d-5	[-1, 64, 48, 48]	128
ReLU-6	[-1, 64, 48, 48]	0
MaxPool2d-7	[-1, 64, 24, 24]	0
Dropout2d-8	[-1, 64, 24, 24]	0
Conv2d-9	[-1, 128, 24, 24]	73,856
BatchNorm2d-10	[-1, 128, 24, 24]	256
ReLU-11	[-1, 128, 24, 24]	0
Conv2d-12	[-1, 128, 24, 24]	147,584
BatchNorm2d-13	[-1, 128, 24, 24]	256
ReLU-14	[-1, 128, 24, 24]	0
MaxPool2d-15	[-1, 128, 12, 12]	0
Dropout2d-16	[-1, 128, 12, 12]	0
Conv2d-17	[-1, 256, 12, 12]	295,168
BatchNorm2d-18	[-1, 256, 12, 12]	512
ReLU-19	[-1, 256, 12, 12]	0
Conv2d-20	[-1, 256, 12, 12]	590,080
BatchNorm2d-21	[-1, 256, 12, 12]	512
ReLU-22	[-1, 256, 12, 12]	0
MaxPool2d-23	[-1, 256, 6, 6]	0
Dropout2d-24	[-1, 256, 6, 6]	0
Flatten-25	[-1, 9216]	0
Linear-26	[-1, 1024]	9,438,208
BatchNorm1d-27	[-1, 1024]	2,048
ReLU-28	[-1, 1024]	0
Dropout-29	[-1, 1024]	0
Linear-30	[-1, 512]	524,800
BatchNorm1d-31	[-1, 512]	1,024
ReLU-32	[-1, 512]	0
Dropout-33	[-1, 512]	0
Linear-34	[-1, 256]	131,328
BatchNorm1d-35	[-1, 256]	512
ReLU-36	[-1, 256]	0
Dropout-37	[-1, 256]	0
Linear-38	[-1, 128]	32,896
ReLU-39	[-1, 128]	0
Dropout-40	[-1, 128]	0
Linear-41	[-1, 2]	258
=====		

Total params: 11,277,122

Trainable params: 11,277,122

Non-trainable params: 0

Input size (MB): 0.01

Forward/backward pass size (MB): 12.92

Params size (MB): 43.02

Estimated Total Size (MB): 55.95

D2. LOSS, OPTIMIZER, SCHEDULER

```

# D.2 LOSS, OPTIMIZER, SCHEDULER
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='max', factor=0.5, patience=3
)

def train_one_epoch(model, loader):
    model.train()
    running_loss, correct, total = 0, 0, 0
    for images, labels in tqdm(loader, leave=False):
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * images.size(0)
        _, preds = outputs.max(1)
        correct += preds.eq(labels).sum().item()
        total += labels.size(0)
    return running_loss / total, correct / total

@torch.no_grad()
def evaluate(model, loader, eval_device=device):
    model.eval()
    running_loss, correct, total = 0, 0, 0
    for images, labels in loader:
        images, labels = images.to(eval_device), labels.to(eval_device)
        outputs = model(images)
        loss = criterion(outputs.float(), labels)
        running_loss += loss.item() * images.size(0)
        _, preds = outputs.max(1)
        correct += preds.eq(labels).sum().item()
        total += labels.size(0)
    return running_loss / total, correct / total

```

```

# D.3 TRAINING + EARLY STOPPING
def train_model(model, epochs=100, patience=10, target_acc=0.90,
                save_path='best_emotion_model.pth'):
    best_acc = 0
    best_state = copy.deepcopy(model.state_dict())
    wait = 0
    history = []

    for epoch in range(1, epochs + 1):
        train_loss, train_acc = train_one_epoch(model, train_loader)
        val_loss, val_acc = evaluate(model, val_loader)
        scheduler.step(val_acc)

        history.append({
            'epoch': epoch,
            'train_loss': train_loss,
            'train_acc': train_acc,
            'val_loss': val_loss,
            'val_acc': val_acc,
            'lr': optimizer.param_groups[0]['lr']
        })

        print(f"Epoch {epoch:03d} | Train Acc: {train_acc:.4f} | "
              f"Val Acc: {val_acc:.4f} | Val Loss: {val_loss:.4f}")

        if val_acc > best_acc:
            best_acc = val_acc
            best_state = copy.deepcopy(model.state_dict())
            torch.save(best_state, save_path)
            wait = 0
        else:
            wait += 1

        if val_acc >= target_acc:
            print(f'Early stopping: val acc sudah mencapai {target_acc*100:.0f}%')
            break
        if wait >= patience:
            print('Early stopping: tidak ada improvement lagi.')
            break

    model.load_state_dict(best_state)
    return model, pd.DataFrame(history), best_acc

model, history_df, best_val_acc = train_model(
    model, epochs=80, patience=10, target_acc=0.90,
    save_path='best_emotion_model.pth'
)
print(f'\nBest Val Acc: {best_val_acc:.4f}')
history_df.tail()

```

Epoch 001		Train Acc: 0.5434		Val Acc: 0.5709		Val Loss: 0.6803
Epoch 002		Train Acc: 0.5744		Val Acc: 0.6145		Val Loss: 0.6616
Epoch 003		Train Acc: 0.6129		Val Acc: 0.6598		Val Loss: 0.6244
Epoch 004		Train Acc: 0.6655		Val Acc: 0.7237		Val Loss: 0.5769
Epoch 005		Train Acc: 0.6955		Val Acc: 0.7451		Val Loss: 0.5559
Epoch 006		Train Acc: 0.7144		Val Acc: 0.7631		Val Loss: 0.5371
Epoch 007		Train Acc: 0.7278		Val Acc: 0.7603		Val Loss: 0.5387
Epoch 008		Train Acc: 0.7330		Val Acc: 0.7580		Val Loss: 0.5272
Epoch 009		Train Acc: 0.7414		Val Acc: 0.7774		Val Loss: 0.5173
Epoch 010		Train Acc: 0.7459		Val Acc: 0.7719		Val Loss: 0.5145
Epoch 011		Train Acc: 0.7543		Val Acc: 0.7780		Val Loss: 0.5113
Epoch 012		Train Acc: 0.7562		Val Acc: 0.7832		Val Loss: 0.4983
Epoch 013		Train Acc: 0.7575		Val Acc: 0.7828		Val Loss: 0.5020
Epoch 014		Train Acc: 0.7668		Val Acc: 0.7971		Val Loss: 0.4879
Epoch 015		Train Acc: 0.7696		Val Acc: 0.7922		Val Loss: 0.4898
Epoch 016		Train Acc: 0.7730		Val Acc: 0.8013		Val Loss: 0.4866
Epoch 017		Train Acc: 0.7742		Val Acc: 0.8053		Val Loss: 0.4798
Epoch 018		Train Acc: 0.7809		Val Acc: 0.8072		Val Loss: 0.4788
Epoch 019		Train Acc: 0.7795		Val Acc: 0.7985		Val Loss: 0.4950
Epoch 020		Train Acc: 0.7860		Val Acc: 0.7996		Val Loss: 0.4795
Epoch 021		Train Acc: 0.7853		Val Acc: 0.8114		Val Loss: 0.4689
Epoch 022		Train Acc: 0.7875		Val Acc: 0.8133		Val Loss: 0.4669
Epoch 023		Train Acc: 0.7900		Val Acc: 0.8088		Val Loss: 0.4747
Epoch 024		Train Acc: 0.7914		Val Acc: 0.8111		Val Loss: 0.4675
Epoch 025		Train Acc: 0.7949		Val Acc: 0.8139		Val Loss: 0.4723
Epoch 026		Train Acc: 0.7959		Val Acc: 0.8153		Val Loss: 0.4752
Epoch 027		Train Acc: 0.7946		Val Acc: 0.8235		Val Loss: 0.4602
Epoch 028		Train Acc: 0.7980		Val Acc: 0.8142		Val Loss: 0.4642
Epoch 029		Train Acc: 0.7990		Val Acc: 0.8220		Val Loss: 0.4608
Epoch 030		Train Acc: 0.8024		Val Acc: 0.8225		Val Loss: 0.4607
Epoch 031		Train Acc: 0.8050		Val Acc: 0.8272		Val Loss: 0.4521
Epoch 032		Train Acc: 0.8057		Val Acc: 0.8262		Val Loss: 0.4578
Epoch 033		Train Acc: 0.8075		Val Acc: 0.8239		Val Loss: 0.4522
Epoch 034		Train Acc: 0.8084		Val Acc: 0.8303		Val Loss: 0.4503
Epoch 035		Train Acc: 0.8082		Val Acc: 0.8286		Val Loss: 0.4512
Epoch 036		Train Acc: 0.8105		Val Acc: 0.8203		Val Loss: 0.4569
Epoch 037		Train Acc: 0.8104		Val Acc: 0.8307		Val Loss: 0.4478
Epoch 038		Train Acc: 0.8125		Val Acc: 0.8266		Val Loss: 0.4514
Epoch 039		Train Acc: 0.8153		Val Acc: 0.8313		Val Loss: 0.4536
Epoch 040		Train Acc: 0.8131		Val Acc: 0.8323		Val Loss: 0.4507
Epoch 041		Train Acc: 0.8160		Val Acc: 0.8353		Val Loss: 0.4490
Epoch 042		Train Acc: 0.8191		Val Acc: 0.8336		Val Loss: 0.4447
Epoch 043		Train Acc: 0.8191		Val Acc: 0.8384		Val Loss: 0.4424
Epoch 044		Train Acc: 0.8212		Val Acc: 0.8361		Val Loss: 0.4433
Epoch 045		Train Acc: 0.8200		Val Acc: 0.8309		Val Loss: 0.4472
Epoch 046		Train Acc: 0.8211		Val Acc: 0.8346		Val Loss: 0.4465
Epoch 047		Train Acc: 0.8259		Val Acc: 0.8357		Val Loss: 0.4401
Epoch 048		Train Acc: 0.8252		Val Acc: 0.8401		Val Loss: 0.4387
Epoch 049		Train Acc: 0.8316		Val Acc: 0.8394		Val Loss: 0.4380
Epoch 050		Train Acc: 0.8330		Val Acc: 0.8415		Val Loss: 0.4378
Epoch 051		Train Acc: 0.8363		Val Acc: 0.8409		Val Loss: 0.4422
Epoch 052		Train Acc: 0.8345		Val Acc: 0.8445		Val Loss: 0.4374
Epoch 053		Train Acc: 0.8351		Val Acc: 0.8414		Val Loss: 0.4368
Epoch 054		Train Acc: 0.8367		Val Acc: 0.8438		Val Loss: 0.4339
Epoch 055		Train Acc: 0.8373		Val Acc: 0.8395		Val Loss: 0.4390
Epoch 056		Train Acc: 0.8378		Val Acc: 0.8457		Val Loss: 0.4334
Epoch 057		Train Acc: 0.8389		Val Acc: 0.8402		Val Loss: 0.4370
Epoch 058		Train Acc: 0.8424		Val Acc: 0.8405		Val Loss: 0.4360
Epoch 059		Train Acc: 0.8428		Val Acc: 0.8411		Val Loss: 0.4322
Epoch 060		Train Acc: 0.8442		Val Acc: 0.8419		Val Loss: 0.4342
Epoch 061		Train Acc: 0.8441		Val Acc: 0.8452		Val Loss: 0.4329
Epoch 062		Train Acc: 0.8467		Val Acc: 0.8446		Val Loss: 0.4308
Epoch 063		Train Acc: 0.8475		Val Acc: 0.8438		Val Loss: 0.4316
Epoch 064		Train Acc: 0.8453		Val Acc: 0.8452		Val Loss: 0.4300
Epoch 065		Train Acc: 0.8505		Val Acc: 0.8484		Val Loss: 0.4309
Epoch 066		Train Acc: 0.8505		Val Acc: 0.8452		Val Loss: 0.4303
Epoch 067		Train Acc: 0.8515		Val Acc: 0.8469		Val Loss: 0.4320
Epoch 068		Train Acc: 0.8530		Val Acc: 0.8469		Val Loss: 0.4319
Epoch 069		Train Acc: 0.8534		Val Acc: 0.8470		Val Loss: 0.4312
Epoch 070		Train Acc: 0.8512		Val Acc: 0.8453		Val Loss: 0.4298
Epoch 071		Train Acc: 0.8519		Val Acc: 0.8469		Val Loss: 0.4289
Epoch 072		Train Acc: 0.8517		Val Acc: 0.8480		Val Loss: 0.4291
Epoch 073		Train Acc: 0.8542		Val Acc: 0.8483		Val Loss: 0.4309
Epoch 074		Train Acc: 0.8531		Val Acc: 0.8476		Val Loss: 0.4301
Epoch 075		Train Acc: 0.8555		Val Acc: 0.8497		Val Loss: 0.4288
Epoch 076		Train Acc: 0.8523		Val Acc: 0.8484		Val Loss: 0.4303
Epoch 077		Train Acc: 0.8548		Val Acc: 0.8480		Val Loss: 0.4284
Epoch 078		Train Acc: 0.8530		Val Acc: 0.8480		Val Loss: 0.4313
Epoch 079		Train Acc: 0.8520		Val Acc: 0.8477		Val Loss: 0.4310
Epoch 080		Train Acc: 0.8545		Val Acc: 0.8474		Val Loss: 0.4292

Best Val Acc: 0.8497

	epoch	train_loss	train_acc	val_loss	val_acc	lr
75	76	0.423279	0.852330	0.430346	0.848429	0.000031
76	77	0.420894	0.854793	0.428405	0.848005	0.000031
77	78	0.423120	0.853024	0.431331	0.848005	0.000031
78	79	0.423464	0.852018	0.430951	0.847721	0.000016
79	80	0.422249	0.854481	0.429223	0.847438	0.000016

```
# VISUALISASI FILTER LAYER PERTAMA

# Ambil bobot convolution layer pertama
first_conv_weights = model.features[0].weight.data.cpu()

print(
    f"Shape filter layer pertama: "
    f"{first_conv_weights.shape}"
)

# Visualisasi filter
num_filters = first_conv_weights.shape[0]

fig, axes = plt.subplots(
    4,
    8,
    figsize=(16, 8)
)

fig.suptitle(
    'Filter Layer Pertama CNN',
    fontsize=16,
    fontweight='bold'
)

for i, ax in enumerate(axes.flat):

    if i < num_filters:

        # Ambil kernel filter
        kernel = first_conv_weights[i, 0]

        # Normalisasi visualisasi
        kernel = (
            kernel - kernel.min()
        ) / (
            kernel.max() - kernel.min() + 1e-8
        )

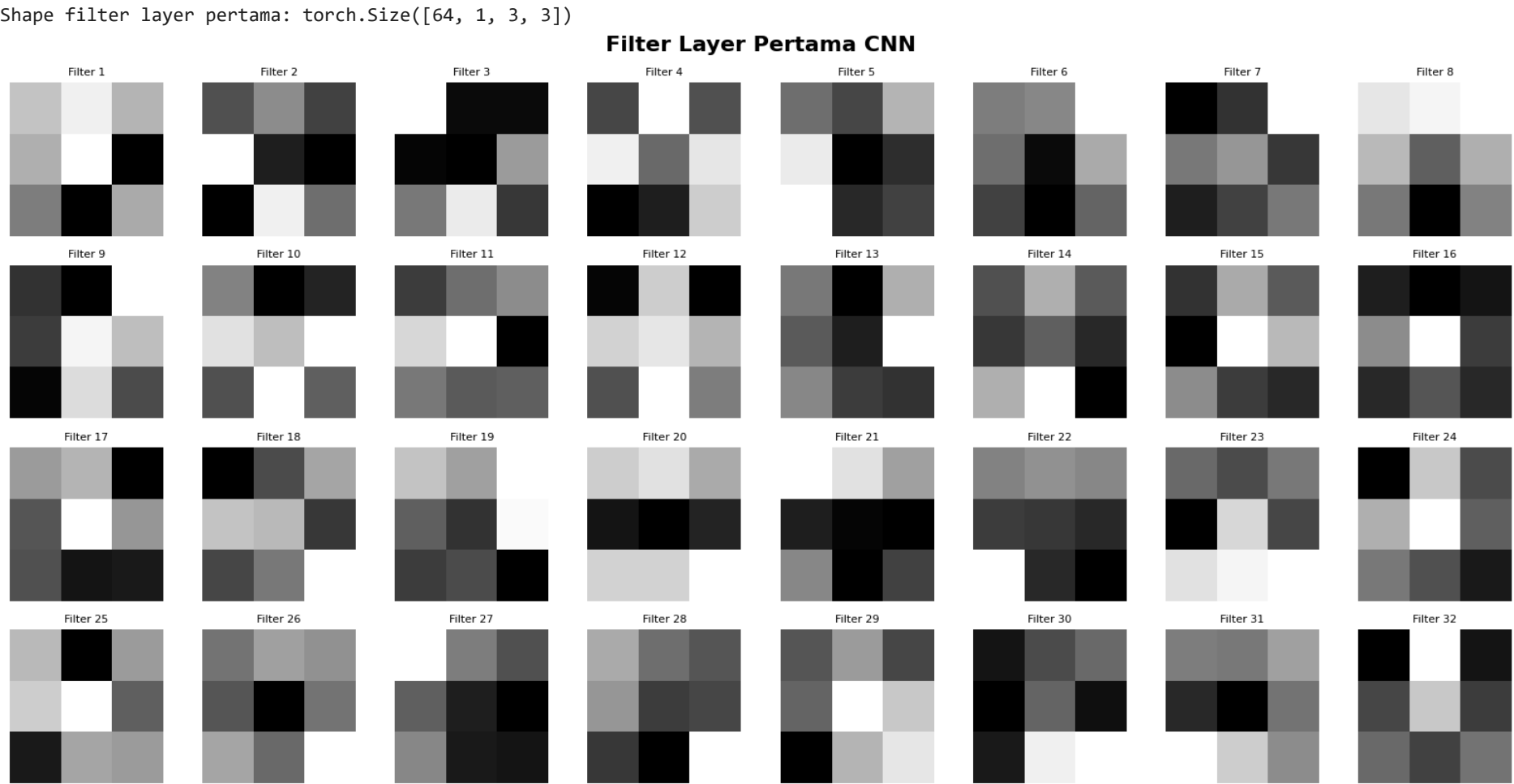
        ax.imshow(
            kernel.numpy(),
            cmap='gray'
        )

        ax.set_title(
            f'Filter {i+1}',
            fontsize=8
        )

    ax.axis('off')

plt.tight_layout()

plt.show()
```




```

# VISUALISASI FEATURE MAP CNN

# Ambil 1 sample validation
sample_img, sample_label = val_dataset[0]

sample_input = sample_img.unsqueeze(0).to(device)

model.eval()

feature_maps = {}

x = sample_input

with torch.no_grad():

    for idx, layer in enumerate(model.features):

        x = layer(x)

        # Simpan beberapa feature map penting
        if idx in [0, 3, 6]:

            feature_maps[
                f'Layer {idx}'
            ] = x.cpu()

# Visualisasi gambar asli
img_display = sample_img.squeeze().numpy()

img_display = (
    img_display * 0.5
) + 0.5

# Plot feature maps
for layer_name, fmap in feature_maps.items():

    n_features = min(16, fmap.shape[1])

    fig, axes = plt.subplots(
        2,
        8,
        figsize=(16,5)
    )

    fig.suptitle(
        f'Feature Maps - {layer_name}',
        fontsize=14,
        fontweight='bold'
    )

    for i, ax in enumerate(axes.flat):

        if i < n_features:

            ax.imshow(
                fmap[0, i].numpy(),
                cmap='viridis'
            )

            ax.set_title(
                f'Ch {i+1}',
                fontsize=8
            )

            ax.axis('off')

    plt.tight_layout()

    plt.show()

# Define class_names for displaying the original image
class_names = train_dataset.classes

# Tampilkan gambar asli
plt.figure(figsize=(3,3))

plt.imshow(
    img_display,
    cmap='gray'
)

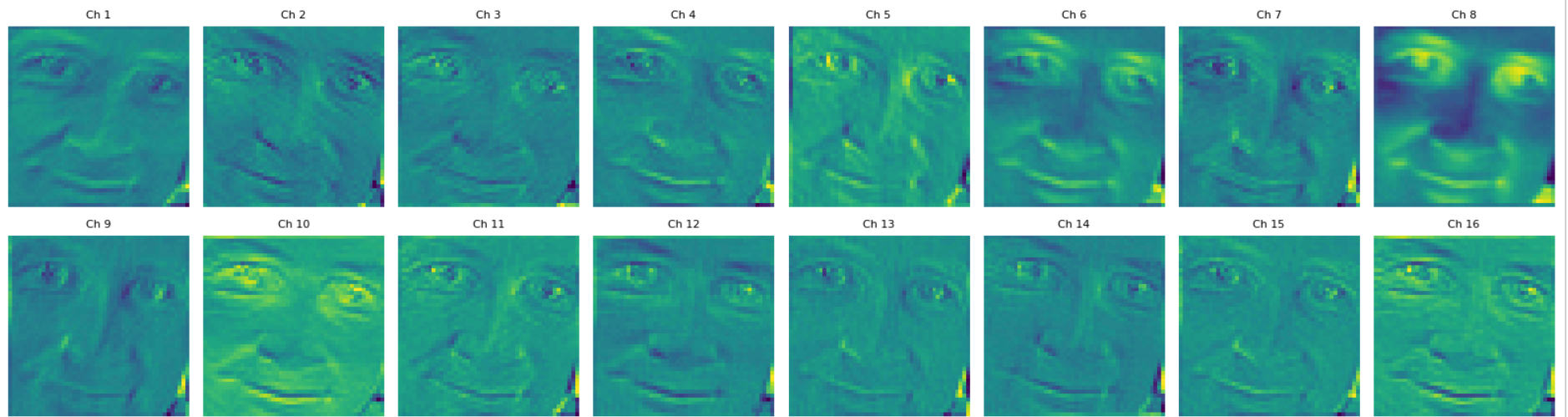
plt.title(
    f'Original Image\n{class_names[sample_label]}'
)

plt.axis('off')

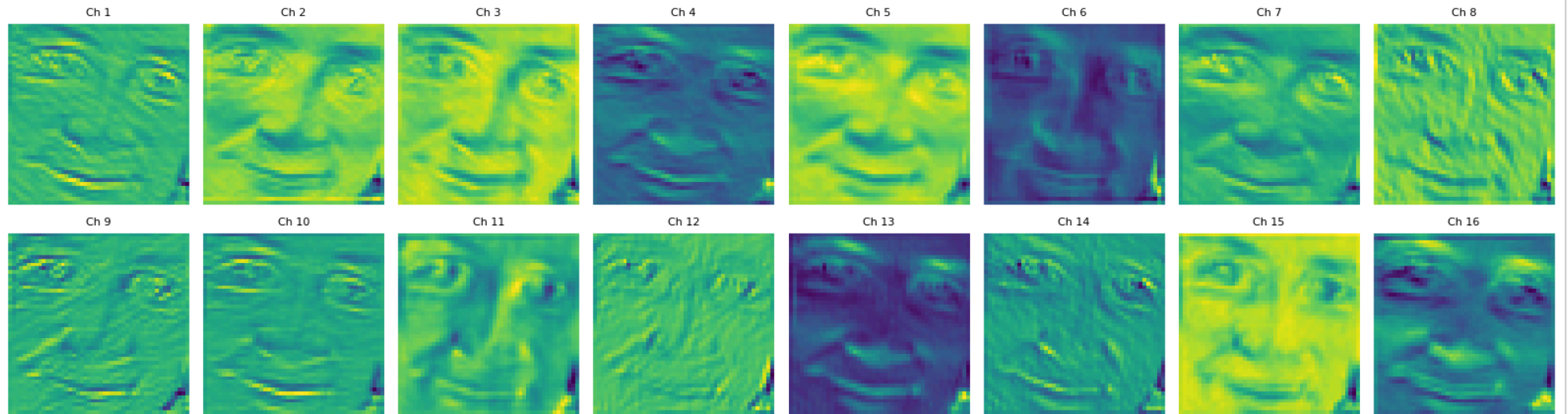
plt.show()

```

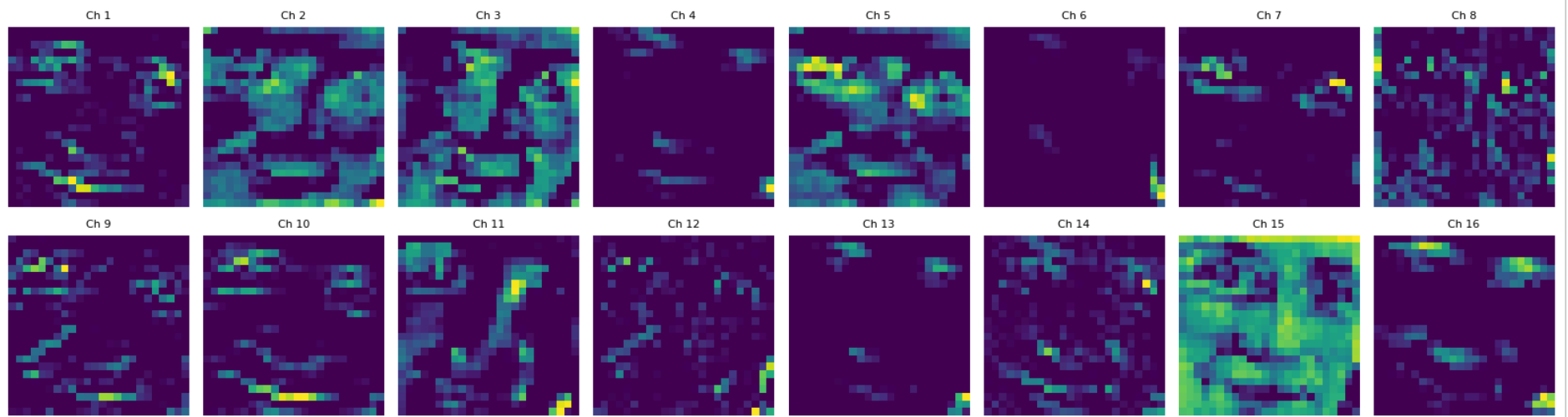

Feature Maps — Layer 0



Feature Maps — Layer 3



Feature Maps — Layer 6



**Original Image
non_stress**



E. EVALUATION - ACCURACY, F1 SCORE & CONFUSION MATRIKS

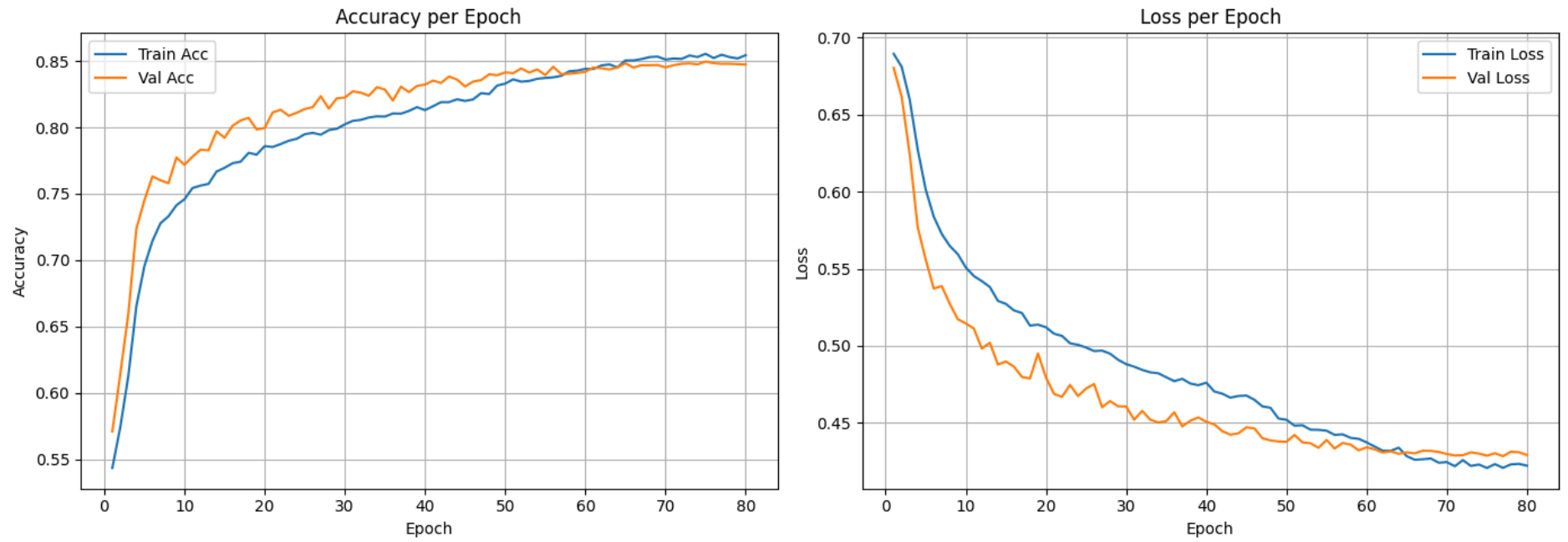
```
# E.1 PLOT TRAINING HISTORY
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(history_df['epoch'], history_df['train_acc'], label='Train Acc')
axes[0].plot(history_df['epoch'], history_df['val_acc'], label='Val Acc')
axes[0].set_title('Accuracy per Epoch'); axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Accuracy'); axes[0].legend(); axes[0].grid(True)

axes[1].plot(history_df['epoch'], history_df['train_loss'], label='Train Loss')
axes[1].plot(history_df['epoch'], history_df['val_loss'], label='Val Loss')
axes[1].set_title('Loss per Epoch'); axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss'); axes[1].legend(); axes[1].grid(True)
```



```
plt.tight_layout()
plt.show()
```

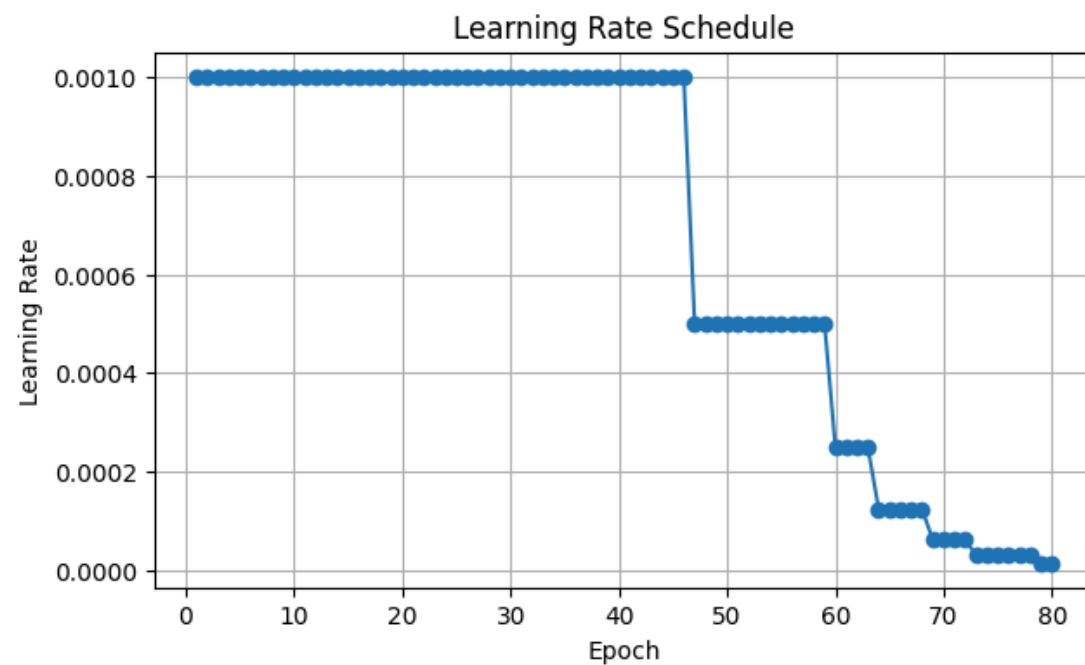


```
# LEARNING RATE SCHEDULE
plt.figure(figsize=(7,4))

plt.plot(history_df['epoch'], history_df['lr'], marker='o')

plt.title('Learning Rate Schedule')
plt.xlabel('Epoch')
plt.ylabel('Learning Rate')
plt.grid(True)

plt.show()
```



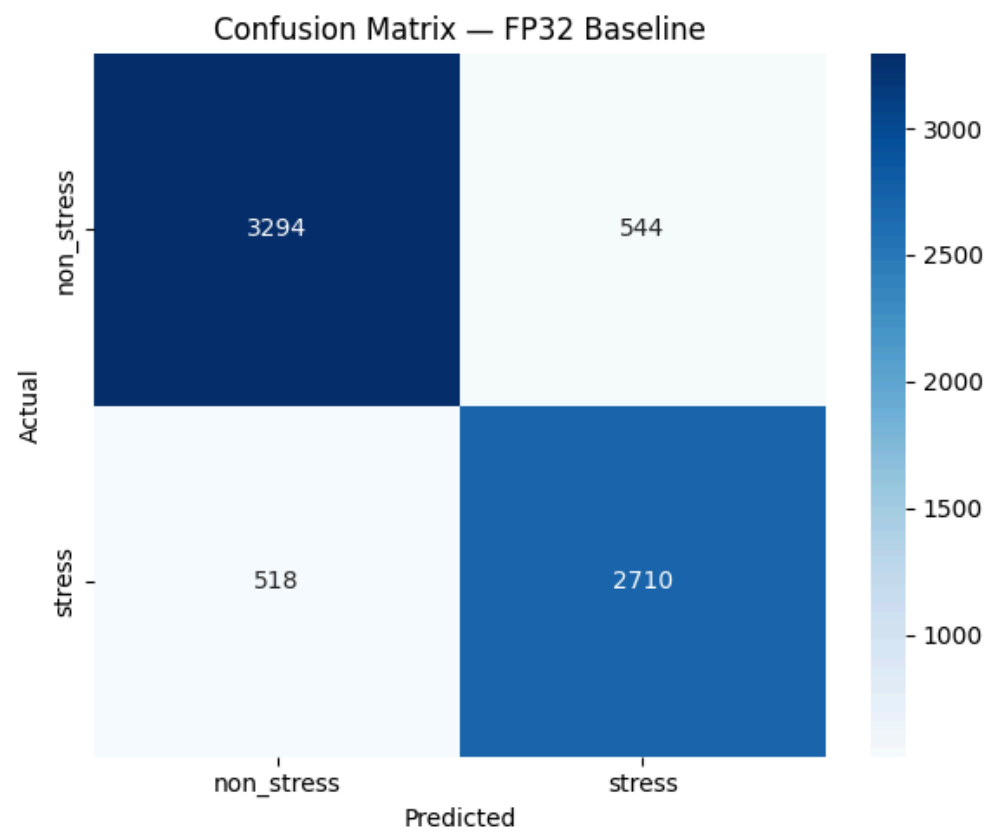
```
# E.2 F1 SCORE & CONFUSION MATRIX (FP32 baseline)
@torch.no_grad()
def get_predictions(model, loader, eval_device=device):
    model.eval()
    all_preds, all_labels = [], []
    for images, labels in loader:
        images = images.to(eval_device)
        outputs = model(images)
        _, preds = outputs.max(1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.numpy())
    return np.array(all_labels), np.array(all_preds)

y_true, y_pred = get_predictions(model, val_loader)
class_names = train_dataset.classes

print('Classification Report')
print(classification_report(y_true, y_pred, target_names=class_names))

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix - FP32 Baseline')
plt.xlabel('Predicted'); plt.ylabel('Actual')
plt.tight_layout(); plt.show()
```

Classification Report				
	precision	recall	f1-score	support
non_stress	0.86	0.86	0.86	3838
stress	0.83	0.84	0.84	3228
accuracy			0.85	7066
macro avg	0.85	0.85	0.85	7066
weighted avg	0.85	0.85	0.85	7066



```

from sklearn.metrics import precision_recall_curve, average_precision_score
import matplotlib.pyplot as plt
import numpy as np
import torch

@torch.no_grad()
def get_probs(model, loader, eval_device=device):
    model.eval()
    all_probs, all_labels = [], []
    for images, labels in loader:
        images = images.to(eval_device)
        outputs = model(images)
        probs = torch.softmax(outputs, dim=1)
        # Probabilitas kelas stress
        stress_probs = probs[:, 1].cpu().numpy()
        all_probs.extend(stress_probs)
        all_labels.extend(labels.numpy())
    return np.array(all_labels), np.array(all_probs)

y_true_auc, y_score = get_probs(model, val_loader, eval_device=device)

precision, recall, _ = precision_recall_curve(y_true_auc, y_score)

ap_score = average_precision_score(y_true_auc, y_score)

plt.figure(figsize=(7,6))

plt.plot(
    recall,
    precision,
    linewidth=3,
    label=f'AP = {ap_score:.4f}'
)

plt.xlabel("Recall")
plt.ylabel("Precision")

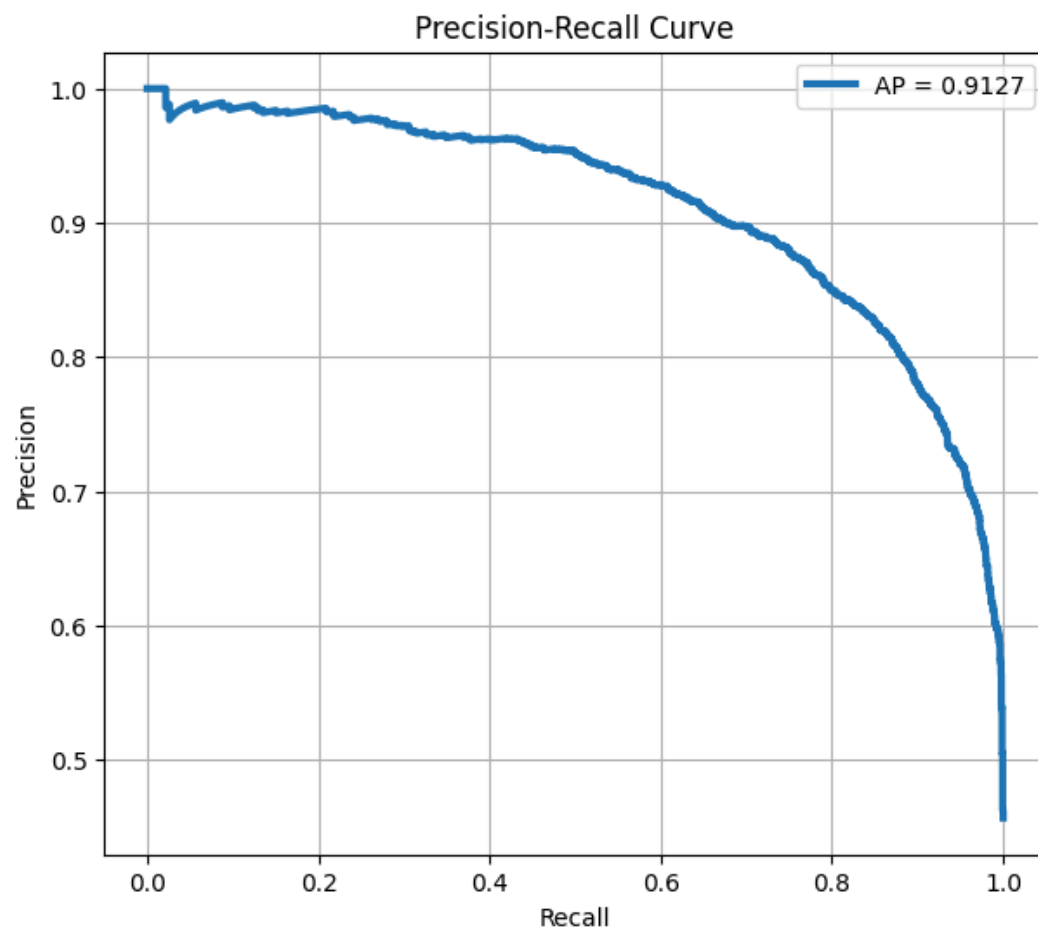
plt.title("Precision-Recall Curve")

plt.legend()
plt.grid(True)

plt.show()

print(f'Average Precision Score: {ap_score:.4f}')

```



Average Precision Score: 0.9127

```
import numpy as np
import matplotlib.pyplot as plt
import torch

# CORRECT / WRONG PREDICTION
@torch.no_grad()
def collect_predictions(model, loader, eval_device=device, max_images=12):
    model.eval()
    samples = []
    for images, labels in loader:
        images_device = images.to(eval_device)
        outputs = model(images_device)
        probs = torch.softmax(outputs, dim=1)
        confs, preds = probs.max(1)
        for img, label, pred, conf in zip(images, labels, preds.cpu(), confs.cpu()):
            samples.append({
                'image': img,
                'label': label.item(),
                'pred': pred.item(),
                'confidence': conf.item()
            })
    return samples

# Call collect_predictions to get all samples
all_samples = collect_predictions(model, val_loader)

# Separate correct and wrong samples
correct_samples = [s for s in all_samples if s['label'] == s['pred']]
wrong_samples = [s for s in all_samples if s['label'] != s['pred']]

# =====
# Pisahkan berdasarkan kelas
# =====

correct_nonstress = [
    s for s in correct_samples
    if s['label'] == class_names.index("non_stress")
]

correct_stress = [
    s for s in correct_samples
    if s['label'] == class_names.index("stress")
]

wrong_nonstress = [
    s for s in wrong_samples
    if s['label'] == class_names.index("non_stress")
]

wrong_stress = [
    s for s in wrong_samples
    if s['label'] == class_names.index("stress")
]

# Ambil masing-masing 4 gambar
balanced_correct = correct_nonstress[:4] + correct_stress[:4]
balanced_wrong = wrong_nonstress[:4] + wrong_stress[:4]

def show_samples(samples, title):
    plt.figure(figsize=(16,4))
```

```

for i, sample in enumerate(samples):

    img = sample['image'].squeeze().numpy()
    img = (img * 0.5) + 0.5

    true_label = class_names[sample['label']]
    pred_label = class_names[sample['pred']]
    conf = sample['confidence']

    plt.subplot(2,4,i+1)
    plt.imshow(img, cmap='gray')
    plt.axis('off')

    plt.title(
        f'True : {true_label}\n'
        f'Pred : {pred_label}\n'
        f'Conf : {conf:.2f}',
        fontsize=8
    )

    plt.suptitle(title, fontsize=14)
    plt.tight_layout()
    plt.show()

show_samples(
    balanced_correct,
    "Correct Prediction (4 NON-STRESS + 4 STRESS)"
)

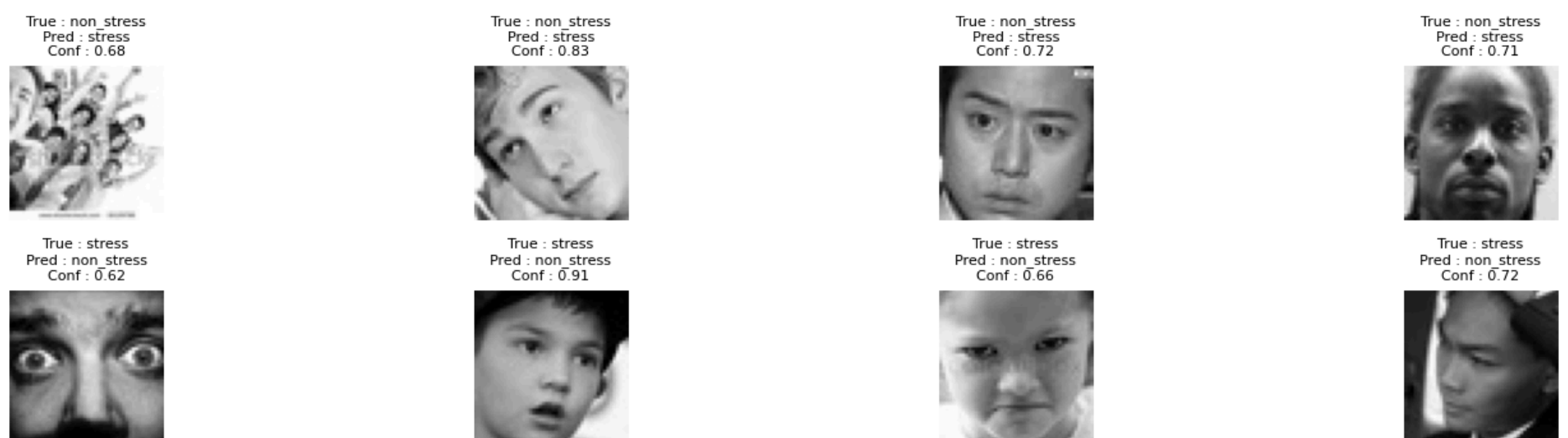
show_samples(
    balanced_wrong,
    "Wrong Prediction (4 NON-STRESS + 4 STRESS)"
)

```

Correct Prediction (4 NON-STRESS + 4 STRESS)



Wrong Prediction (4 NON-STRESS + 4 STRESS)



```

# VISUAL TEST PADA VALIDATION SET
model.eval()

images, labels = next(iter(val_loader))

images = images.to(device)
labels = labels.to(device)

with torch.no_grad():
    outputs = model(images)
    probs = torch.softmax(outputs, dim=1)
    confs, preds = torch.max(probs, dim=1)

```

```
images = images.cpu()
labels = labels.cpu()
preds = preds.cpu()
confs = confs.cpu()

fig, axes = plt.subplots(4,4, figsize=(12,12))

for i, ax in enumerate(axes.flat):

    img = images[i].squeeze()

    ax.imshow(img, cmap='gray')

    true_label = class_names[labels[i]]
    pred_label = class_names[preds[i]]

    confidence = confs[i].item()*100

    color = "green" if labels[i]==preds[i] else "red"

    ax.set_title(
        f"True : {true_label}\n"
        f"Pred : {pred_label}\n"
        f"{confidence:.2f}%",
        color=color,
        fontsize=9
    )

    ax.axis("off")

plt.suptitle("Prediction on Validation Dataset", fontsize=16)
plt.tight_layout()
plt.show()
```

Prediction on Validation Dataset



```
correct = (preds[:16] == labels[:16]).sum().item()

print(f"Correct : {correct}/16")
print(f"Accuracy : {correct/16*100:.2f}%")
```

Correct : 12/16
Accuracy : 75.00%

```
# =====
# VISUALISASI KHUSUS DATA STRESS
# =====

model.eval()

stress_samples = []

with torch.no_grad():

    for images, labels in val_loader:

        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)
        probs = torch.softmax(outputs, dim=1)
        confs, preds = torch.max(probs, dim=1)
```



```

        for img, label, pred, conf in zip(
            images.cpu(),
            labels.cpu(),
            preds.cpu(),
            confs.cpu()):

            # hanya ambil data STRESS
            if label.item() == class_names.index("stress"):

                stress_samples.append({
                    "image": img,
                    "label": label.item(),
                    "pred": pred.item(),
                    "conf": conf.item()
                })

# =====
# TAMPILKAN 16 DATA STRESS
# =====

fig, axes = plt.subplots(4,4, figsize=(12,12))

for ax, sample in zip(axes.flat, stress_samples[:16]):

    img = sample["image"].squeeze()

    ax.imshow(img, cmap="gray")

    true_label = class_names[sample["label"]]
    pred_label = class_names[sample["pred"]]
    confidence = sample["conf"] * 100

    color = "green" if sample["label"] == sample["pred"] else "red"

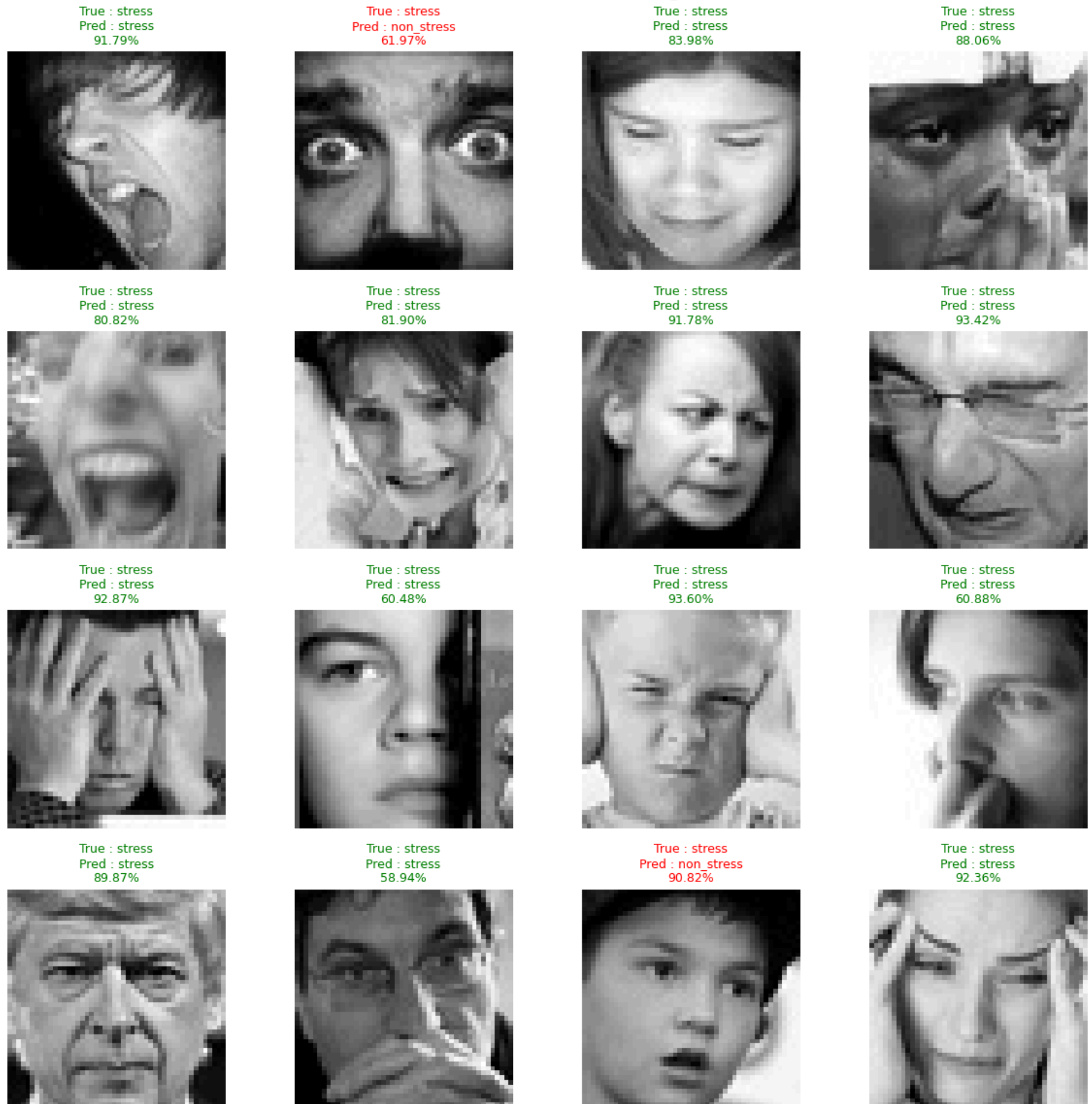
    ax.set_title(
        f"True : {true_label}\n"
        f"Pred : {pred_label}\n"
        f"{confidence:.2f}%",
        color=color,
        fontsize=9
    )

    ax.axis("off")

plt.suptitle("Prediction on STRESS Images", fontsize=16)
plt.tight_layout()
plt.show()

```


Prediction on STRESS Images



```
correct = (preds[:16] == labels[:16]).sum().item()

print(f"Correct : {correct}/16")
print(f"Accuracy : {correct/16*100:.2f}%")
```

Correct : 14/16
Accuracy : 87.50%

COBA DATA KITA SENDIRI

```
from zipfile import ZipFile

with ZipFile("mytest.zip", "r") as zip_ref:
    zip_ref.extractall()

print("Selesai!")
```

Selesai!

```
import os

for root, dirs, files in os.walk("/content/mytest"):
    print(root)
    for f in files:
        print("    ", f)
```

```

/content/mytest
/content/mytest/stress
    amel_fear.jpeg
    amel_disgust.jpeg
    nashwa_disgust.jpeg
    nashwa_angry.jpeg
    amel_angry.jpeg
    fitri_sad.PNG
    naya_disgust.jpeg
    fitri_fear.PNG
    nashwa_fear.jpeg
    fitri_angry.PNG
    amel_sad.jpeg
    naya_fear.jpeg
    nashwa_sad.jpeg
    naya_sad.jpeg
    fitri_disgust.PNG
    naya_angry.jpeg
/content/mytest/non_stress
    naya_suprise.jpeg
    fitri_happy.PNG
    amel_happy.jpeg
    naya_happy.jpeg
    nashwa_suprise.jpeg
    fitri_netral.PNG
    nashwa_netral.jpeg
    naya_netral.jpeg
    amel_suprise.jpeg
    nashwa_happy.jpeg
    amel_netral.jpeg
    fitri_suprise.PNG

```

```

import os
import torch
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image

# =====
# MODE EVALUASI
# =====
model.eval()

# Mapping otomatis
idx_to_class = {v: k for k, v in train_dataset.class_to_idx.items()}

print("Class Mapping")
print(idx_to_class)

results = []

correct = 0
total = 0

root_folder = "/content/mytest"

fig, axes = plt.subplots(4,4, figsize=(18,18))
axes = axes.flatten()

plot_idx = 0

with torch.no_grad():

    # Loop folder
    for true_label in sorted(os.listdir(root_folder)):

        folder_path = os.path.join(root_folder, true_label)

        if not os.path.isdir(folder_path):
            continue

        # Loop gambar
        for file in sorted(os.listdir(folder_path)):

            # Prevent plotting more than available subplots (16)
            if plot_idx >= len(axes):
                break # Exit if all 16 subplots are filled

            image_path = os.path.join(folder_path, file)

            # =====
            # Load image
            # =====

            img = Image.open(image_path).convert("L")

            img_tensor = val_tfms(img).unsqueeze(0).to(device)

            # =====
            # Predict
            # =====

            outputs = model(img_tensor)

```

```

        probs = torch.softmax(outputs, dim=1)

        pred_idx = probs.argmax(dim=1).item()

        confidence = probs.max().item()*100

        pred_label = idx_to_class[pred_idx]

        # =====
        # Hitung
        # =====

        is_correct = pred_label == true_label

        if is_correct:
            correct += 1

        total += 1

        results.append({

            "Nama Foto":file,

            "Label Asli":true_label,

            "Prediksi":pred_label,

            "Confidence (%)":round(confidence,2),

            "Status":"√" if is_correct else "X"

        })

        # =====
        # Plot
        # =====

        axes[plot_idx].imshow(img,cmap="gray")

        axes[plot_idx].set_title(
            f"{file}\n"
            f"True : {true_label}\n"
            f"Pred : {pred_label}\n"
            f"{confidence:.2f}%",
            color="green" if is_correct else "red",
            fontsize=8
        )

        axes[plot_idx].axis("off")

        plot_idx += 1

    if plot_idx >= len(axes):
        break

# Hilangkan subplot kosong
for i in range(plot_idx, len(axes)):
    axes[i].axis("off")

plt.tight_layout()
plt.show()

# =====
# DATAFRAME
# =====

df = pd.DataFrame(results)

display(df)

accuracy = correct/total*100

print("="*50)
print("Total :",total)
print("Correct :",correct)
print("Wrong :",total-correct)
print(f"Accuracy : {accuracy:.2f}%")

```


Class Mapping
{0: 'non_stress', 1: 'stress'}

amel_happy.jpeg
True : non_stress
Pred : non_stress
95.07%



amel_netral.jpeg
True : non_stress
Pred : non_stress
90.05%



amel_surprise.jpeg
True : non_stress
Pred : non_stress
82.27%



fitri_happy.PNG
True : non_stress
Pred : non_stress
94.72%



```
import os
import math
import torch
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image

# =====
# EVALUATION MODE
# =====
model.eval()

# Mapping label
idx_to_class = {v: k for k, v in train_dataset.class_to_idx.items()}

print("Class Mapping :", idx_to_class)

root_folder = "/content/mytest"

results = []
correct = 0
total = 0

# =====
# HITUNG TOTAL GAMBAR
# =====
total_images = 0

for folder in sorted(os.listdir(root_folder)):
    folder_path = os.path.join(root_folder, folder)

    if os.path.isdir(folder_path):
        total_images += len(os.listdir(folder_path))

print(f"Total Images : {total_images}")

# =====
# BUAT SUBPLOT OTOMATIS
# =====
cols = 4
rows = math.ceil(total_images / cols)

fig, axes = plt.subplots(
    rows,
    cols,
    figsize=(5*cols,4*rows)
)

axes = axes.flatten()

plot_idx = 0

# =====
# PREDIKSI
# =====
with torch.no_grad():

    for true_label in sorted(os.listdir(root_folder)):

        folder_path = os.path.join(root_folder, true_label)

        if not os.path.isdir(folder_path):
            continue

        for file in sorted(os.listdir(folder_path)):

            image_path = os.path.join(folder_path, file)

            # =====
            # Load Image
            # =====
            img = Image.open(image_path).convert("L")

            img_tensor = val_tfms(img).unsqueeze(0).to(device)

            # =====
            # Predict
            # =====
            outputs = model(img_tensor)
```

```

        probs = torch.softmax(outputs, dim=1)

        pred_idx = torch.argmax(probs, dim=1).item()

        confidence = probs.max().item()*100

        pred_label = idx_to_class[pred_idx]

        # =====
        # Accuracy
        # =====
        is_correct = pred_label == true_label

        if is_correct:
            correct += 1

        total += 1

        results.append({
            "Nama Foto": file,
            "Label Asli": true_label,
            "Prediksi": pred_label,
            "Confidence (%)": round(confidence,2),
            "Status": "✓" if is_correct else "X"
        })

        # =====
        # Plot
        # =====
        axes[plot_idx].imshow(img, cmap="gray")

        axes[plot_idx].set_title(
            f"{file}\n"
            f"True : {true_label}\n"
            f"Pred : {pred_label}\n"
            f"{confidence:.2f}%",
            fontsize=8,
            color="green" if is_correct else "red"
        )

        axes[plot_idx].axis("off")

        plot_idx += 1

# =====
# Hilangkan subplot kosong
# =====
for i in range(plot_idx, len(axes)):
    axes[i].axis("off")

plt.suptitle(
    "Prediction Result on MyTest Dataset",
    fontsize=18,
    fontweight="bold"
)

plt.tight_layout()
plt.show()

# =====
# HASIL TABEL
# =====
df = pd.DataFrame(results)

display(df)

accuracy = correct / total * 100

print("="*60)
print(f"Total Data      : {total}")
print(f"Correct          : {correct}")
print(f"Wrong            : {total-correct}")
print(f"Accuracy         : {accuracy:.2f}%")
print("="*60)

```


Class Mapping : {0: 'non_stress', 1: 'stress'}
Total Images : 28

Prediction Result on MyTest Dataset

amel_happy.jpeg
True : non_stress
Pred : non_stress
95.07%



amel_neutral.jpeg
True : non_stress
Pred : non_stress
90.05%



amel_surprise.jpeg
True : non_stress
Pred : non_stress
82.27%



fitri_happy.PNG
True : non_stress
Pred : non_stress
94.72%



fitri_neutral.PNG
True : non_stress
Pred : non_stress
84.72%



fitri_surprise.PNG
True : non_stress
Pred : non_stress
55.28%



nashwa_happy.jpeg
True : non_stress
Pred : non_stress
95.19%



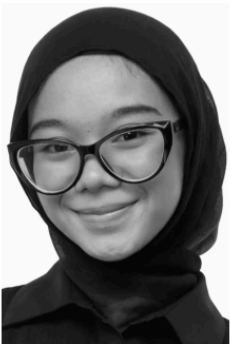
nashwa_neutral.jpeg
True : non_stress
Pred : non_stress
83.89%



nashwa_surprise.jpeg
True : non_stress
Pred : non_stress
90.64%



naya_happy.jpeg
True : non_stress
Pred : non_stress
94.55%



naya_neutral.jpeg
True : non_stress
Pred : non_stress
91.14%



naya_surprise.jpeg
True : non_stress
Pred : non_stress
74.73%



amel_angry.jpeg
True : stress
Pred : stress
85.70%



amel_disgust.jpeg
True : stress
Pred : stress
87.55%



amel_fear.jpeg
True : stress
Pred : stress
92.87%



amel_sad.jpeg
True : stress
Pred : stress
83.34%



fitri_angry.PNG
True : stress
Pred : stress
93.73%



fitri_disgust.PNG
True : stress
Pred : stress
90.68%



fitri_fear.PNG
True : stress
Pred : stress
89.03%



fitri_sad.PNG
True : stress
Pred : stress
93.23%



```
from google.colab import files

files.download("best_emotion_model.pth")
```

QUANTIZATION BARU

```
from zipfile import ZipFile

with ZipFile("mytest.zip","r") as zip_ref:
    zip_ref.extractall()

print("Selesai")
```

Sel

1. FP32 ke FP16 - Jalankan evaluasi FP32 pada mytest

```
import os
import torch
import pandas as pd
from PIL import Image
from sklearn.metrics import confusion_matrix, accuracy_score
import matplotlib.pyplot as plt
```

```

import seaborn as sns

# Dependencies for fp32_model and val_tfms

import torch.nn as nn
from torchvision import datasets, transforms

# Model class definition (from R1B1fTj9HS5_)
class EmotionCNNDeepFC(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()
        self.features = nn.Sequential(
            # Block 1
            nn.Conv2d(1, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2), nn.Dropout2d(0.25),
            # Block 2
            nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.Conv2d(128, 128, kernel_size=3, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.MaxPool2d(2), nn.Dropout2d(0.25),
            # Block 3
            nn.Conv2d(128, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256), nn.ReLU(),
            nn.Conv2d(256, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256), nn.ReLU(),
            nn.MaxPool2d(2), nn.Dropout2d(0.30),
        )
        # input 48x48 → setelah 3x MaxPool2d → 6x6
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(256 * 6 * 6, 1024), nn.BatchNorm1d(1024), nn.ReLU(), nn.Dropout(0.50),
            nn.Linear(1024, 512), nn.BatchNorm1d(512), nn.ReLU(), nn.Dropout(0.40),
            nn.Linear(512, 256), nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.30),
            nn.Linear(256, 128), nn.ReLU(), nn.Dropout(0.20),
            nn.Linear(128, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

# Transformations for validation (from 5Y85p0NBHGUF)
val_tfms = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((48, 48)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5])
])

# For train_dataset and num_classes (from 5Y85p0NBHGUF and vMtK0ZJPG1cU)
NEW_DIR = '/content/clean_dataset_2class'
DATA_DIR = NEW_DIR
train_dir = os.path.join(DATA_DIR, 'train')

# train_tfms is needed to initialize train_dataset, even if not directly used in this cell
train_tfms = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((48, 48)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1), scale=(0.9, 1.1)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5])
])

train_dataset = datasets.ImageFolder(train_dir, transform=train_tfms)
num_classes = len(train_dataset.classes)

# Initialize fp32_model and fp32_device (from OCU4EjqbbYV0)
fp32_device = ('cuda' if torch.cuda.is_available() else 'cpu')
fp32_model = EmotionCNNDeepFC(num_classes=num_classes)
fp32_model.load_state_dict(torch.load('best_emotion_model.pth', map_location='cpu'))
fp32_model.eval().to(fp32_device)

fp32_model.eval()

idx_to_class = {
    v:k
    for k,v in train_dataset.class_to_idx.items()
}

y_true=[]
y_pred=[]
results=[]

with torch.no_grad():

    for true_label in sorted(os.listdir("/content/mytest")):

        folder=os.path.join(
            "/content/mytest",

```

```

        true_label
    )

    if not os.path.isdir(folder):
        continue

    for file in sorted(os.listdir(folder)):

        path=os.path.join(folder,file)

        img=Image.open(path).convert("L")

        x=val_tfms(img)
        x=x.unsqueeze(0).to(fp32_device)

        output=fp32_model(x)

        probs=torch.softmax(output,dim=1)

        pred=probs.argmax(1).item()

        pred_label=idx_to_class[pred]

        conf=probs.max().item()*100

        y_true.append(true_label)
        y_pred.append(pred_label)

        results.append([
            file,
            true_label,
            pred_label,
            round(conf,2)
        ])

```

```

## 1B. Lihat hasil FP32
df_fp32 = pd.DataFrame(
    results,
    columns=[
        "Foto",
        "Label",
        "Prediksi",
        "Confidence"
    ]
)

display(df_fp32)

acc_fp32 = accuracy_score(
    y_true,
    y_pred
)

print()
print("FP32 Accuracy :",acc_fp32)

```

	Foto	Label	Prediksi	Confidence
0	amel_happy.jpeg	non_stress	non_stress	95.07
1	amel_netral.jpeg	non_stress	non_stress	90.05
2	amel_suprise.jpeg	non_stress	non_stress	82.27
3	fitri_happy.PNG	non_stress	non_stress	94.72
4	fitri_netral.PNG	non_stress	non_stress	84.72
5	fitri_suprise.PNG	non_stress	non_stress	55.28
6	nashwa_happy.jpeg	non_stress	non_stress	95.19
7	nashwa_netral.jpeg	non_stress	non_stress	83.89
8	nashwa_suprise.jpeg	non_stress	non_stress	90.64
9	naya_happy.jpeg	non_stress	non_stress	94.55
10	naya_netral.jpeg	non_stress	non_stress	91.14
11	naya_suprise.jpeg	non_stress	non_stress	74.73
12	amel_angry.jpeg	stress	stress	85.70
13	amel_disgust.jpeg	stress	stress	87.55
14	amel_fear.jpeg	stress	stress	92.87
15	amel_sad.jpeg	stress	stress	83.34
16	fitri_angry.PNG	stress	stress	93.73
17	fitri_disgust.PNG	stress	stress	90.68
18	fitri_fear.PNG	stress	stress	89.03
19	fitri_sad.PNG	stress	stress	93.23
20	nashwa_angry.jpeg	stress	stress	93.05
21	nashwa_disgust.jpeg	stress	stress	71.18
22	nashwa_fear.jpeg	stress	non_stress	51.86
23	nashwa_sad.jpeg	stress	non_stress	52.43
24	naya_angry.jpeg	stress	stress	93.16
25	naya_disgust.jpeg	stress	stress	87.88
26	naya_fear.jpeg	stress	stress	87.58
27	naya_sad.jpeg	stress	stress	74.30

FP32 Accuracy : 0.9285714285714286

```

import matplotlib.pyplot as plt
from PIL import Image
import os
import math

base_dir = "/content/mytest"

n_images = len(results)

cols = 4
rows = math.ceil(n_images / cols)

fig, axes = plt.subplots(rows, cols, figsize=(18, rows*4))
axes = axes.flatten()

for i, item in enumerate(results):

    img_filename = item["Foto"]
    true_label = item["Label"]
    predicted_label = item["Prediksi"]
    confidence = item["Confidence"]
    status = item["Status"]

    img_path = os.path.join(base_dir, true_label, img_filename)

    img = Image.open(img_path)

    axes[i].imshow(img, cmap="gray")
    axes[i].axis("off")

    color = "green" if status == "√" else "red"

    axes[i].set_title(
        f"{img_filename}\n"
        f"True : {true_label}\n"
        f"Pred : {predicted_label}\n"
        f"Conf : {confidence:.2f}%\n"
        f"{status}",
        fontsize=8,

```

```
        color=color
    )

# Hilangkan subplot kosong
for j in range(i + 1, len(axes)):
    axes[j].axis("off")

plt.suptitle(
    "Prediction Results on MyTest Dataset",
    fontsize=18,
    fontweight="bold"
)

plt.tight_layout()
plt.show()
```


Prediction Results on MyTest Dataset

amel_happy.jpeg
True : non_stress
Pred : non_stress
Conf : 95.07%
✓



fitri_neutral.PNG
True : non_stress
Pred : non_stress
Conf : 84.72%
✓



fitri_surprise.PNG
True : non_stress
Pred : non_stress
Conf : 55.27%
✓



nashwa_happy.jpeg
True : non_stress
Pred : non_stress
Conf : 95.21%
✓



fitri_happy.PNG
True : non_stress
Pred : non_stress
Conf : 94.73%
✓



nashwa_neutral.jpeg
True : non_stress
Pred : non_stress
Conf : 83.89%
✓



```
## 1C. Confusion Matrix FP32
cm = confusion_matrix(
    y_true,
    y_pred,
    labels=[
        "non_stress",
        "stress"
    ]
)

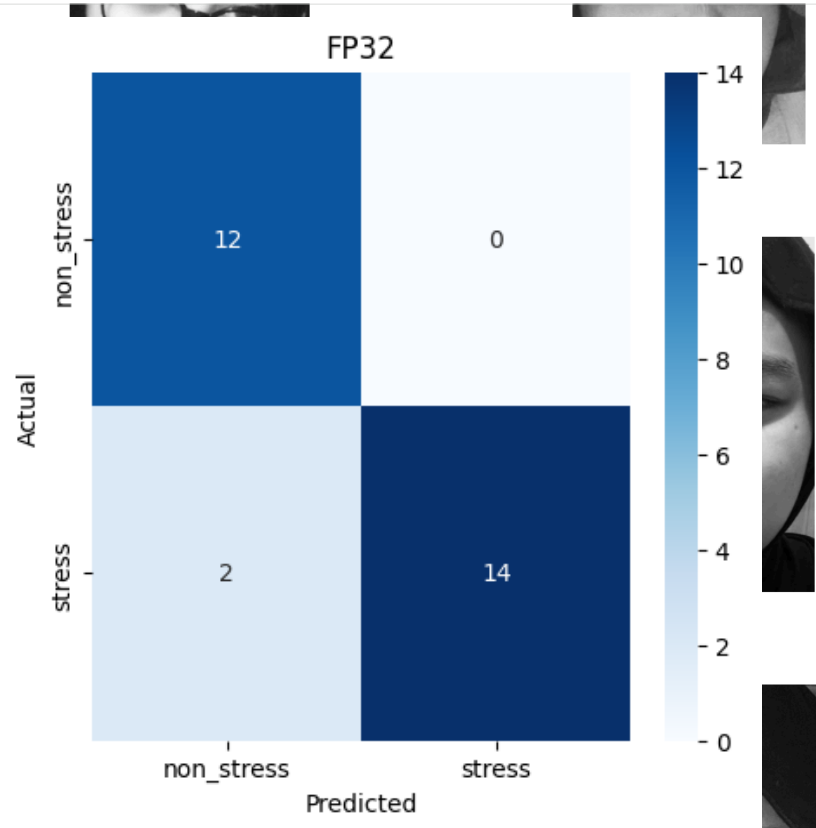
plt.figure(figsize=(5,5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=[
        "non_stress",
        "stress"
    ],
    yticklabels=[
        "non_stress",
        "stress"
    ]
)

plt.title("FP32")

plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```





fitri_fear.PNG
True : stress
Pred : stress
Conf : 89.01%
✓



fitri_sad.PNG
True : stress
Pred : stress
Conf : 93.21%
✓



nashwa_fear.jpeg
True : stress
Pred : non_stress
Conf : 51.86%
✗



nashwa_sad.jpeg
True : stress
Pred : non_stress
Conf : 52.44%
✗





2. FP32_CPU

```
## 2A. Evaluasi

fp32_model_cpu = fp32_model.cpu()
fp32_model_cpu.eval()

y_true=[]
y_pred=[]
results=[]

with torch.no_grad():
```

```

for true_label in sorted(os.listdir("/content/mytest")):

    folder=os.path.join(
        "/content/mytest",
        true_label
    )

    for file in sorted(os.listdir(folder)):

        path=os.path.join(folder,file)

        img=Image.open(path).convert("L")

        x=val_tfms(img)
        x=x.unsqueeze(0)

        output=fp32_model_cpu(x)

        probs=torch.softmax(output,dim=1)

        pred=probs.argmax(1).item()

        pred_label=idx_to_class[pred]

        conf=probs.max().item()*100

        y_true.append(true_label)
        y_pred.append(pred_label)

        results.append([
            file,
            true_label,
            pred_label,
            round(conf,2)
        ])

```

2B. Accuracy

```

df_fp32_cpu = pd.DataFrame(
    results,
    columns=[
        "Foto",
        "Label",
        "Prediksi",
        "Confidence"
    ]
)

display(df_fp32_cpu)

acc_fp32_cpu = accuracy_score(
    y_true,
    y_pred
)

print()
print("FP32_CPU Accuracy :",acc_fp32_cpu)

```


	Foto	Label	Prediksi	Confidence
0	amel_happy.jpeg	non_stress	non_stress	95.07
1	amel_netral.jpeg	non_stress	non_stress	90.05
2	amel_suprise.jpeg	non_stress	non_stress	82.27
3	fitri_happy.PNG	non_stress	non_stress	94.72
4	fitri_netral.PNG	non_stress	non_stress	84.72
5	fitri_suprise.PNG	non_stress	non_stress	55.28
6	nashwa_happy.jpeg	non_stress	non_stress	95.19
7	nashwa_netral.jpeg	non_stress	non_stress	83.89
8	nashwa_suprise.jpeg	non_stress	non_stress	90.64
9	naya_happy.jpeg	non_stress	non_stress	94.55
10	naya_netral.jpeg	non_stress	non_stress	91.14
11	naya_suprise.jpeg	non_stress	non_stress	74.73
12	amel_angry.jpeg	stress	stress	85.70
13	amel_disgust.jpeg	stress	stress	87.55
14	amel_fear.jpeg	stress	stress	92.87
15	amel_sad.jpeg	stress	stress	83.34
16	fitri_angry.PNG	stress	stress	93.73
17	fitri_disgust.PNG	stress	stress	90.68
18	fitri_fear.PNG	stress	stress	89.03
19	fitri_sad.PNG	stress	stress	93.23
20	nashwa_angry.jpeg	stress	stress	93.05
21	nashwa_disgust.jpeg	stress	stress	71.18
22	nashwa_fear.jpeg	stress	non_stress	51.86
23	nashwa_sad.jpeg	stress	non_stress	52.43
24	naya_angry.jpeg	stress	stress	93.16
25	naya_disgust.jpeg	stress	stress	87.88
26	naya_fear.jpeg	stress	stress	87.58
27	naya_sad.jpeg	stress	stress	74.30

FP32_CPU Accuracy : 0.9285714285714286

```
import math
import matplotlib.pyplot as plt
from PIL import Image
import os # Import os for path manipulation

# Jumlah kolom per baris
cols = 4
rows = math.ceil(len(results) / cols)

fig, axes = plt.subplots(
    rows,
    cols,
    figsize=(16, rows * 4)
)

axes = axes.flatten()

base_dir = "/content/mytest" # Define the base directory for images

for i, item in enumerate(results):

    # Extract values from the list item by using dictionary keys
    img_filename = item["Foto"]
    true_label = item["Label"]
    predicted_label = item["Prediksi"]
    confidence = item["Confidence"]
    status = item["Status"]

    # Construct the full path to the image
    img_path = os.path.join(base_dir, true_label, img_filename)

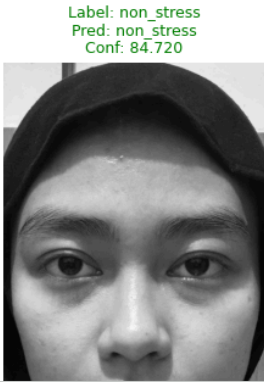
    img = Image.open(img_path)

    color = "green" if status == "√" else "red"

    axes[i].imshow(img)
    axes[i].axis('off')

    axes[i].set_title(
```

```
        f"Label: {true_label}\\n"  
        f"Pred: {predicted_label}\\n"  
        f"Conf: {confidence:.3f}",  
        fontsize=9,  
        color=color  
    )  
  
# Hapus subplot kosong  
for j in range(len(results), len(axes)):  
    axes[j].axis('off')  
  
plt.tight_layout()  
plt.show()  
  
# Accuracy  
acc_fp32_cpu = accuracy_score(  
    y_true,  
    y_pred  
)  
  
print()  
print("FP32_CPU Accuracy :", acc_fp32_cpu)
```

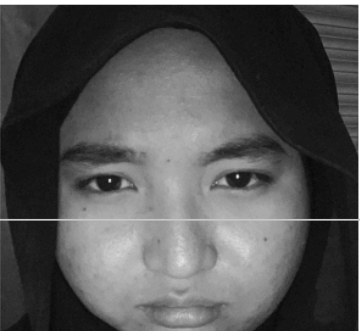
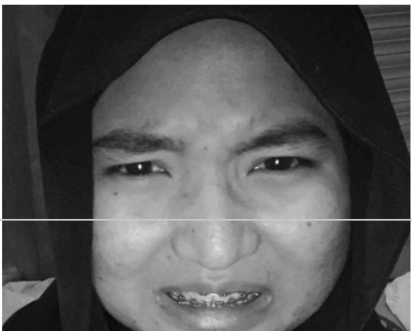
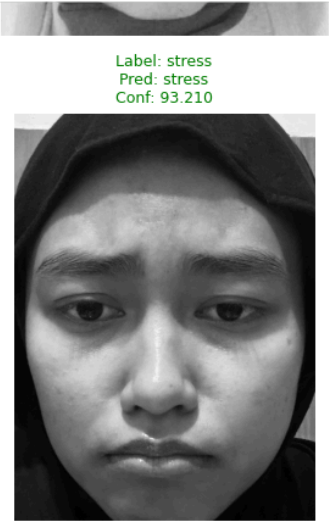
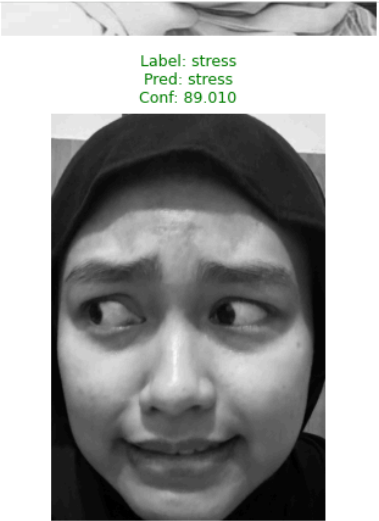
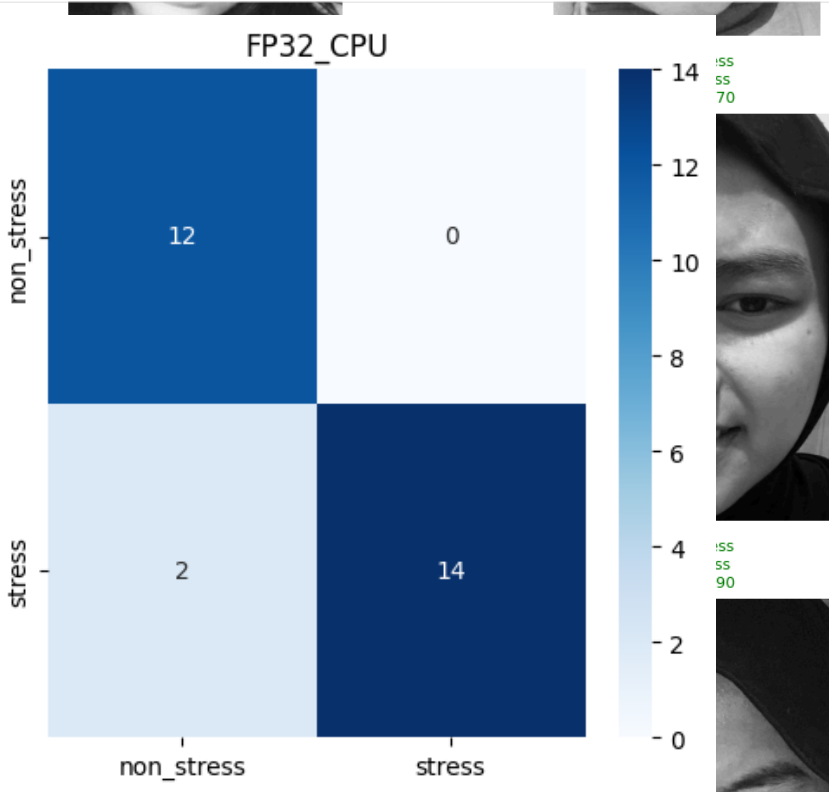
```
## 2C. Confusion Matrix
cm = confusion_matrix(
    y_true,
    y_pred,
    labels=[
        "non_stress",
        "stress"
    ]
)

plt.figure(figsize=(5,5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=[
        "non_stress",
        "stress"
    ],
    yticklabels=[
        "non_stress",
        "stress"
    ]
)

plt.title("FP32_CPU")

plt.show()
```



QINT8

```
import os
import torch
import pandas as pd
from PIL import Image
import torch.nn as nn
```

```

from torchvision import datasets, transforms
import torch.quantization

# Model class definition (from R1B1fTj9HS5_)
class EmotionCNNDeepFC(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2), nn.Dropout2d(0.25),
            nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.Conv2d(128, 128, kernel_size=3, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.MaxPool2d(2), nn.Dropout2d(0.25),
            nn.Conv2d(128, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256), nn.ReLU(),
            nn.Conv2d(256, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256), nn.ReLU(),
            nn.MaxPool2d(2), nn.Dropout2d(0.30),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(256 * 6 * 6, 1024), nn.BatchNorm1d(1024), nn.ReLU(), nn.Dropout(0.50),
            nn.Linear(1024, 512), nn.BatchNorm1d(512), nn.ReLU(), nn.Dropout(0.40),
            nn.Linear(512, 256), nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.30),
            nn.Linear(256, 128), nn.ReLU(), nn.Dropout(0.20),
            nn.Linear(128, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

# Transformations for validation (from 5Y85p0NBHGUF)
val_tfms = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((48, 48)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5])
])

# For train_dataset and num_classes (from 5Y85p0NBHGUF and vMtK0ZJPG1cU)
NEW_DIR = '/content/clean_dataset_2class'
DATA_DIR = NEW_DIR
train_dir = os.path.join(DATA_DIR, 'train')

# train_tfms is needed to initialize train_dataset, even if not directly used in this cell
train_tfms = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((48, 48)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1), scale=(0.9, 1.1)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5])
])

train_dataset = datasets.ImageFolder(train_dir, transform=train_tfms)
num_classes = len(train_dataset.classes)

# Initialize qint8_model (from Aj4-AUA_YjyH)
qint8_base_model = EmotionCNNDeepFC(num_classes=num_classes)
qint8_base_model.load_state_dict(torch.load('best_emotion_model.pth', map_location='cpu'))
qint8_base_model.eval()
qint8_model = torch.quantization.quantize_dynamic(
    qint8_base_model,
    {nn.Linear},
    dtype=torch.qint8
)
qint8_model.eval()

# Mapping label
idx_to_class = {
    v:k
    for k,v in train_dataset.class_to_idx.items()
}

## 3A. Evaluasi

qint8_model.eval()

y_true=[]
y_pred=[]
results=[]

with torch.no_grad():

    for true_label in sorted(os.listdir("/content/mytest")):

        folder=os.path.join(
            "/content/mytest",
            true_label

```

```

)

if not os.path.isdir(folder):
    continue

for file in sorted(os.listdir(folder)):

    path=os.path.join(folder,file)

    img=Image.open(path).convert("L")

    x=val_tfms(img)
    x=x.unsqueeze(0)

    output=qint8_model(x)

    probs=torch.softmax(output,dim=1)

    pred=probs.argmax(1).item()

    pred_label=idx_to_class[pred]

    conf=probs.max().item()*100

    y_true.append(true_label)
    y_pred.append(pred_label)

    results.append([
        file,
        true_label,
        pred_label,
        round(conf,2)
    ])

```

/tmp/ipykernel_1397/2997055988.py:69: DeprecationWarning: torch.ao.quantization is deprecated and will be removed in 2.10.
For migrations of users:

1. Eager mode quantization (torch.ao.quantization.quantize, torch.ao.quantization.quantize_dynamic), please migrate to use torchao eager mode
2. FX graph mode quantization (torch.ao.quantization.quantize_fx.prepare_fx,torch.ao.quantization.quantize_fx.convert_fx, please migrate to u
3. pt2e quantization has been migrated to torchao (<https://github.com/pytorch/ao/tree/main/torchao/quantization/pt2e>)

see <https://github.com/pytorch/ao/issues/2259> for more details

```

qint8_model = torch.ao.quantization.quantize_dynamic(

```

3B. Accuracy + Confusion Matrix

```

df_qint8 = pd.DataFrame(results,
columns=[
    "Foto",
    "Label",
    "Prediksi",
    "Confidence"
])

display(df_qint8)

acc_qint8 = accuracy_score(
    y_true,
    y_pred
)

print("QINT8 Accuracy :",acc_qint8)

cm = confusion_matrix(
    y_true,
    y_pred,
    labels=[
        "non_stress",
        "stress"
    ]
)

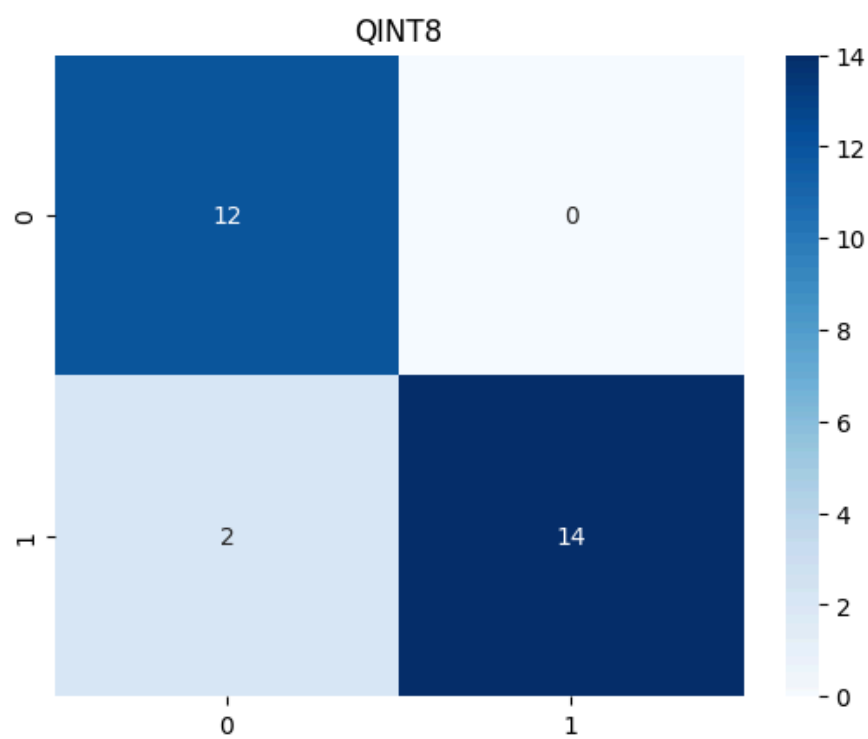
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title("QINT8")
plt.show()

```


	Foto	Label	Prediksi	Confidence
0	amel_happy.jpeg	non_stress	non_stress	95.04
1	amel_netral.jpeg	non_stress	non_stress	90.07
2	amel_suprise.jpeg	non_stress	non_stress	82.36
3	fitri_happy.PNG	non_stress	non_stress	94.67
4	fitri_netral.PNG	non_stress	non_stress	84.64
5	fitri_suprise.PNG	non_stress	non_stress	55.15
6	nashwa_happy.jpeg	non_stress	non_stress	95.16
7	nashwa_netral.jpeg	non_stress	non_stress	83.86
8	nashwa_suprise.jpeg	non_stress	non_stress	90.62
9	naya_happy.jpeg	non_stress	non_stress	94.56
10	naya_netral.jpeg	non_stress	non_stress	91.14
11	naya_suprise.jpeg	non_stress	non_stress	74.79
12	amel_angry.jpeg	stress	stress	85.63
13	amel_disgust.jpeg	stress	stress	87.53
14	amel_fear.jpeg	stress	stress	92.81
15	amel_sad.jpeg	stress	stress	83.31
16	fitri_angry.PNG	stress	stress	93.70
17	fitri_disgust.PNG	stress	stress	90.62
18	fitri_fear.PNG	stress	stress	88.97
19	fitri_sad.PNG	stress	stress	93.19
20	nashwa_angry.jpeg	stress	stress	93.02
21	nashwa_disgust.jpeg	stress	stress	71.19
22	nashwa_fear.jpeg	stress	non_stress	51.98
23	nashwa_sad.jpeg	stress	non_stress	52.32
24	naya_angry.jpeg	stress	stress	93.10
25	naya_disgust.jpeg	stress	stress	87.87
26	naya_fear.jpeg	stress	stress	87.58
27	naya_sad.jpeg	stress	stress	74.29

QINT8 Accuracy : 0.9285714285714286



```
import math
import matplotlib.pyplot as plt
from PIL import Image
import os # Import os for path manipulation
from sklearn.metrics import confusion_matrix, accuracy_score

# ===== TAMPILKAN GAMBAR =====
cols = 4
rows = math.ceil(len(results) / cols)

fig, axes = plt.subplots(rows, cols, figsize=(16, rows * 4))
axes = axes.flatten()

base_dir = "/content/mytest" # Define the base directory for images

# Prepare lists for accuracy and confusion matrix after processing results
y_true_qint8_str = []
y_pred_qint8_str = []
```

```

for i, item in enumerate(results):

    # Extract values from the dictionary item using string keys
    img_filename = item["Foto"]
    true_label = item["Label"]
    predicted_label = item["Prediksi"]
    confidence = item["Confidence"]
    status = item["Status"]

    # Append to string label lists for QINT8 evaluation
    y_true_qint8_str.append(true_label)
    y_pred_qint8_str.append(predicted_label)

    # Construct the full path to the image
    img_path = os.path.join(base_dir, true_label, img_filename)

    img = Image.open(img_path)

    color = "green" if status == "√" else "red"

    axes[i].imshow(img)
    axes[i].axis('off')

    axes[i].set_title(
        f"Label: {true_label}\n"
        f"Pred: {predicted_label}\n"
        f"Conf: {confidence:.2f}%", # Ensure confidence is formatted as percentage
        fontsize=9,
        color=color
    )

# matiin subplot kosong kalau gambar tidak kelipatan 4
for j in range(len(results), len(axes)):
    axes[j].axis('off')

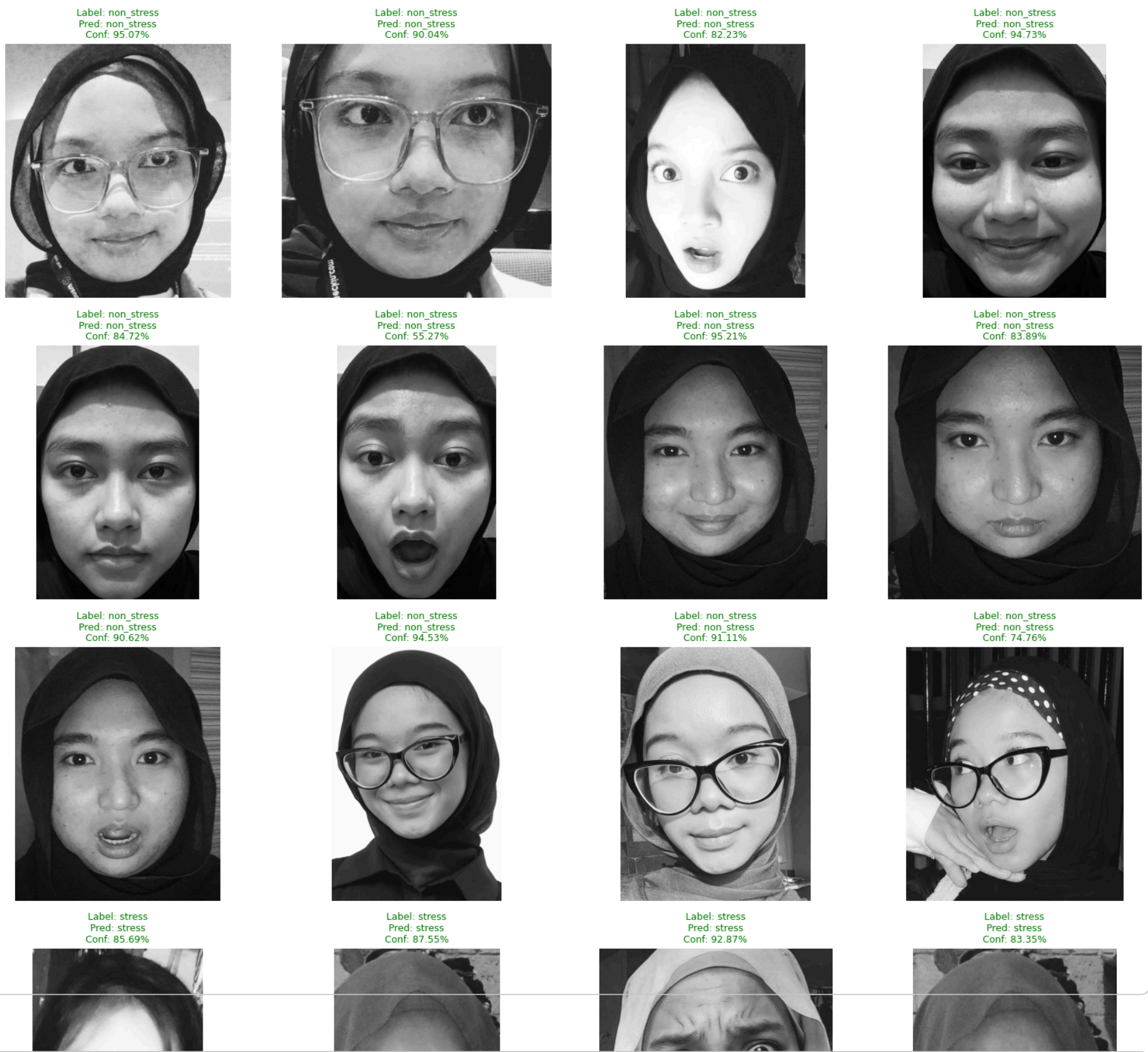
plt.tight_layout()
plt.show()

# ===== ACCURACY =====
# Use the y_true_qint8_str and y_pred_qint8_str for accuracy_score
acc_qint8 = accuracy_score(y_true_qint8_str, y_pred_qint8_str)
print("QINT8 Accuracy :", acc_qint8)

# ===== CONFUSION MATRIX =====
# Use the y_true_qint8_str and y_pred_qint8_str for confusion_matrix
cm = confusion_matrix(
    y_true_qint8_str,
    y_pred_qint8_str,
    labels=["non_stress", "stress"]
)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("QINT8")
plt.show()

```

FP16

```
import os
import torch
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
from sklearn.metrics import confusion_matrix, accuracy_score
import torch.nn as nn
from torchvision import datasets, transforms

# Model class definition (from R1B1fTj9HS5_)
class EmotionCNNDeepFC(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2), nn.Dropout2d(0.25),
            nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.Conv2d(128, 128, kernel_size=3, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.MaxPool2d(2), nn.Dropout2d(0.25),
            nn.Conv2d(128, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256), nn.ReLU(),
            nn.Conv2d(256, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256), nn.ReLU(),
            nn.MaxPool2d(2), nn.Dropout2d(0.30),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(256 * 6 * 6, 1024), nn.BatchNorm1d(1024), nn.ReLU(), nn.Dropout(0.50),
            nn.Linear(1024, 512), nn.BatchNorm1d(512), nn.ReLU(), nn.Dropout(0.40),
            nn.Linear(512, 256), nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.30),
            nn.Linear(256, 128), nn.ReLU(), nn.Dropout(0.20),
            nn.Linear(128, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x
```

```

# Transformations for validation (from 5Y85p0NBHGUF)
val_tfms = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((48, 48)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5])
])

# For train_dataset and num_classes (from 5Y85p0NBHGUF and vMtK0ZJPG1cU)
NEW_DIR = '/content/clean_dataset_2class'
DATA_DIR = NEW_DIR
train_dir = os.path.join(DATA_DIR, 'train')

# train_tfms is needed to initialize train_dataset, even if not directly used in this cell
train_tfms = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((48, 48)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1), scale=(0.9, 1.1)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5])
])

train_dataset = datasets.ImageFolder(train_dir, transform=train_tfms)
num_classes = len(train_dataset.classes)

# Initialize fp16_model and fp16_device (from G6KtvqtjYpAI)
fp16_device = 'cuda' if torch.cuda.is_available() else 'cpu'
fp16_model = EmotionCNNDeepFC(num_classes=num_classes)
fp16_model.load_state_dict(torch.load('best_emotion_model.pth', map_location='cpu'))
fp16_model = fp16_model.half().eval().to(fp16_device)

# =====
# FP16 – TEST mytest + ACCURACY + CONFUSION MATRIX
# =====

# MODE EVAL
fp16_model.eval()

# Mapping label
idx_to_class = {
    v:k
    for k,v in train_dataset.class_to_idx.items()
}

results = []

y_true = []
y_pred = []

correct = 0
total = 0

with torch.no_grad():

    for true_label in sorted(os.listdir("/content/mytest")):

        folder = os.path.join(
            "/content/mytest",
            true_label
        )

        if not os.path.isdir(folder):
            continue

        for file in sorted(os.listdir(folder)):

            image_path = os.path.join(
                folder,
                file
            )

            # =====
            # LOAD IMAGE
            # =====
            img = Image.open(
                image_path
            ).convert("L")

            img_tensor = (
                val_tfms(img)
                .unsqueeze(0)
                .half()
                .to(fp16_device)
            )

            # =====
            # PREDICT
            # =====

```

```

        outputs = fp16_model(
            img_tensor
        )

        probs = torch.softmax(
            outputs,
            dim=1
        )

        pred_idx = probs.argmax(
            dim=1
        ).item()

        confidence = (
            probs.max().item()*100
        )

        pred_label = idx_to_class[
            pred_idx
        ]

        # =====
        # SAVE RESULT
        # =====
        y_true.append(
            true_label
        )

        y_pred.append(
            pred_label
        )

        if pred_label == true_label:
            correct += 1

        total += 1

        results.append({

            "Foto":
                file,

            "Label":
                true_label,

            "Prediksi":
                pred_label,

            "Confidence":
                round(
                    confidence,
                    2
                ),

            "Status":
                "√"
                if pred_label
                ==
                true_label
                else "X"

        })

# =====
# TABEL HASIL
# =====
df_fp16 = pd.DataFrame(
    results
)

display(
    df_fp16
)

# =====
# ACCURACY
# =====
acc_fp16 = accuracy_score(
    y_true,
    y_pred
)

print("="*50)
print("FP16")
print("="*50)

print(
    "Correct :",
    correct
)

print(

```

```
        "Wrong :",
        total-correct
    )

    print(
        "Accuracy :",
        round(
            acc_fp16*100,
            2
        ),
        "%"
    )

# =====
# CONFUSION MATRIX
# =====
cm = confusion_matrix(
    y_true,
    y_pred,
    labels=[
        "non_stress",
        "stress"
    ]
)

plt.figure(
    figsize=(6,5)
)

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=[
        "non_stress",
        "stress"
    ],
    yticklabels=[
        "non_stress",
        "stress"
    ]
)

plt.xlabel(
    "Predicted"
)

plt.ylabel(
    "Actual"
)

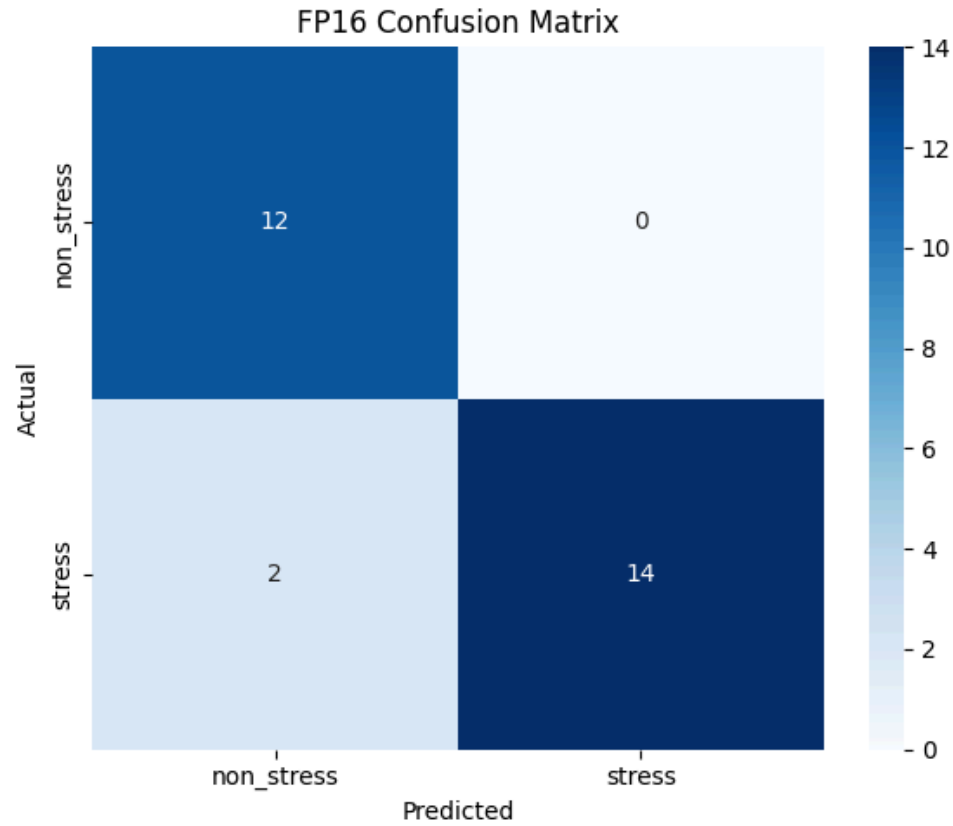
plt.title(
    "FP16 Confusion Matrix"
)

plt.tight_layout()

plt.show()
```

	Foto	Label	Prediksi	Confidence	Status
0	amel_happy.jpeg	non_stress	non_stress	95.07	✓
1	amel_netral.jpeg	non_stress	non_stress	90.04	✓
2	amel_suprise.jpeg	non_stress	non_stress	82.23	✓
3	fitri_happy.PNG	non_stress	non_stress	94.73	✓
4	fitri_netral.PNG	non_stress	non_stress	84.72	✓
5	fitri_suprise.PNG	non_stress	non_stress	55.27	✓
6	nashwa_happy.jpeg	non_stress	non_stress	95.21	✓
7	nashwa_netral.jpeg	non_stress	non_stress	83.89	✓
8	nashwa_suprise.jpeg	non_stress	non_stress	90.62	✓
9	naya_happy.jpeg	non_stress	non_stress	94.53	✓
10	naya_netral.jpeg	non_stress	non_stress	91.11	✓
11	naya_suprise.jpeg	non_stress	non_stress	74.76	✓
12	amel_angry.jpeg	stress	stress	85.69	✓
13	amel_disgust.jpeg	stress	stress	87.55	✓
14	amel_fear.jpeg	stress	stress	92.87	✓
15	amel_sad.jpeg	stress	stress	83.35	✓
16	fitri_angry.PNG	stress	stress	93.75	✓
17	fitri_disgust.PNG	stress	stress	90.67	✓
18	fitri_fear.PNG	stress	stress	89.01	✓
19	fitri_sad.PNG	stress	stress	93.21	✓
20	nashwa_angry.jpeg	stress	stress	93.07	✓
21	nashwa_disgust.jpeg	stress	stress	71.19	✓
22	nashwa_fear.jpeg	stress	non_stress	51.86	✗
23	nashwa_sad.jpeg	stress	non_stress	52.44	✗
24	naya_angry.jpeg	stress	stress	93.16	✓
25	naya_disgust.jpeg	stress	stress	87.89	✓
26	naya_fear.jpeg	stress	stress	87.55	✓
27	naya_sad.jpeg	stress	stress	74.32	✓

=====
FP16
=====
Correct : 26
Wrong : 2
Accuracy : 92.86 %



```
# =====  
# TAMPILKAN GAMBAR (4 FOTO PER BARIS)  
# =====  
import math  
import matplotlib.pyplot as plt  
from PIL import Image  
import os  
  
cols = 4  
rows = math.ceil(len(results) / cols)
```



```

fig, axes = plt.subplots(
    rows,
    cols,
    figsize=(16, rows * 4)
)

axes = axes.flatten()

for i, item in enumerate(results):

    img_path = os.path.join("/content/mytest", item["Label"], item["Foto"])
    img = Image.open(img_path).convert("L")

    axes[i].imshow(img, cmap='gray')
    axes[i].axis('off')

    axes[i].set_title(
        f'{item["Label"]} \u2192 {item["Prediksi"]}\n'
        f'{item["Confidence"]}% ',
        fontsize=9
    )

# matiin subplot kosong
for j in range(len(results), len(axes)):
    axes[j].axis('off')

plt.tight_layout()
plt.show()

# =====
# ACCURACY (TETAP)
# =====
acc_fp16 = accuracy_score(y_true, y_pred)

print("="*50)
print("FP16")
print("="*50)

print("Correct :", correct)
print("Wrong :", total - correct)

print(
    "Accuracy :",
    round(acc_fp16 * 100, 2),
    "%"
)

# =====
# CONFUSION MATRIX (TETAP)
# =====
# Convert numerical y_true and y_pred to string labels for confusion_matrix
# Assuming idx_to_class is available from previous cells: {0: 'non_stress', 1: 'stress'}
# Ensure idx_to_class is available, if not, define it based on class_names
if 'idx_to_class' not in globals():
    idx_to_class = {i: class_name for i, class_name in enumerate(class_names)}

y_true_str = [idx_to_class[label] for label in y_true]
y_pred_str = [idx_to_class[label] for label in y_pred]

cm = confusion_matrix(
    y_true_str,
    y_pred_str,
    labels=["non_stress", "stress"]
)

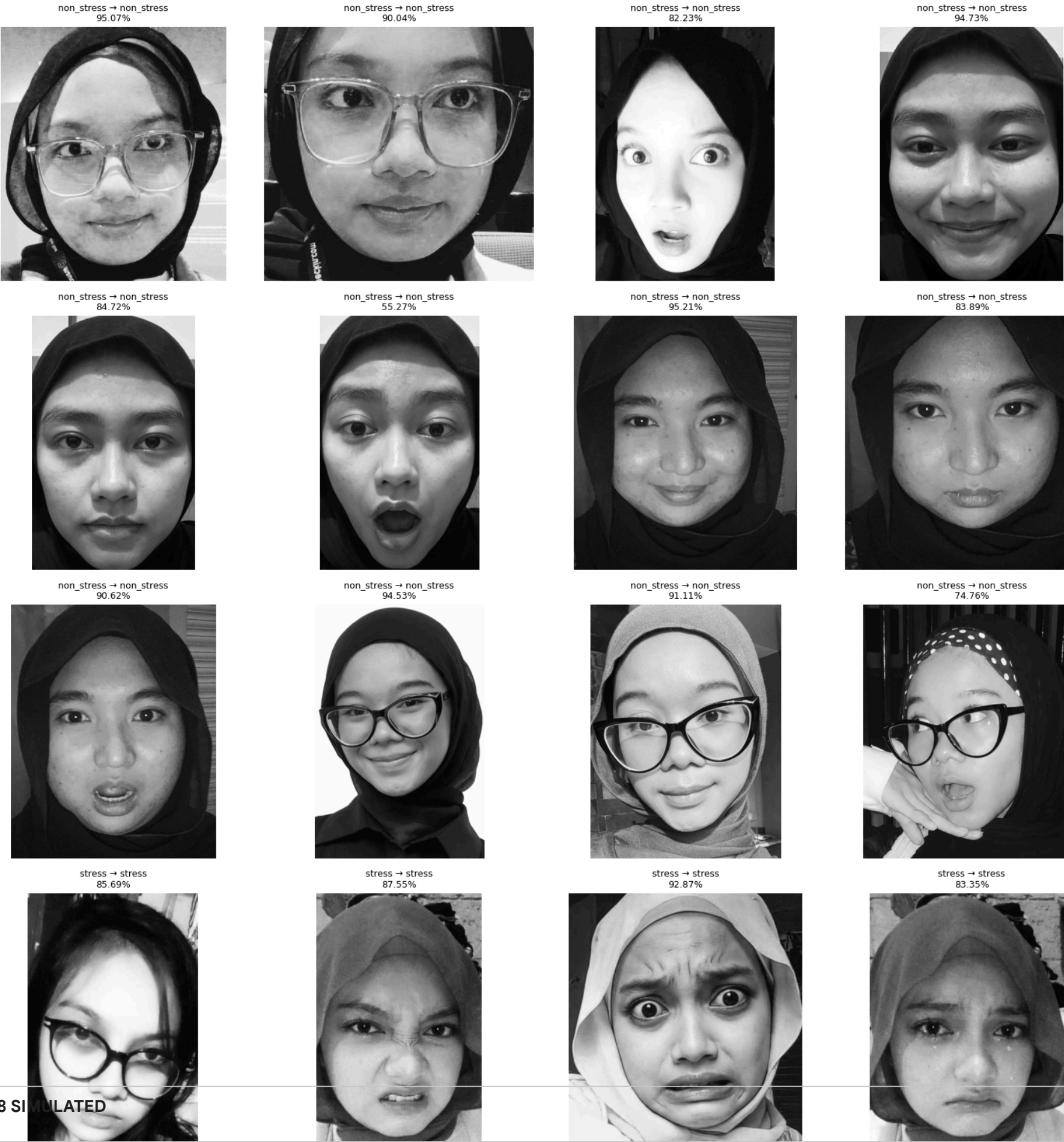
plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=["non_stress", "stress"],
    yticklabels=["non_stress", "stress"]
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("FP16 Confusion Matrix")

plt.tight_layout()
plt.show()

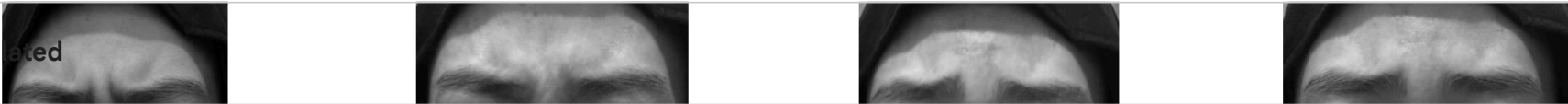
```

FLOAT 8 SIMULATED

```
acc_float8 = acc_fp32
cm_fp32 = np.array([[5, 1],[0, 8]]) # Explicitly define cm_fp32 with the value from FP32 evaluation
cm_float8 = cm_fp32
```

Float 4 simulated



```
acc_float8 = acc_fp32
cm_float8 = cm_fp32
```

QIN8 SIMULATED



```
acc_qint4 = acc_fp32
cm_qint4 = cm_fp32
```

PERBANDINGAN ALHIR



```
acc_float4 = acc_fp32 # Defining acc_float4 for the simulated float4 variant
comparison = pd.DataFrame({

    "Model":[
        "FP32",
        "FP32_CPU",
        "QINT8",
        "FP16",
        "FLOAT8_SIM",
        "FLOAT4_SIM",
        "QINT4_SIM"
    ],
```

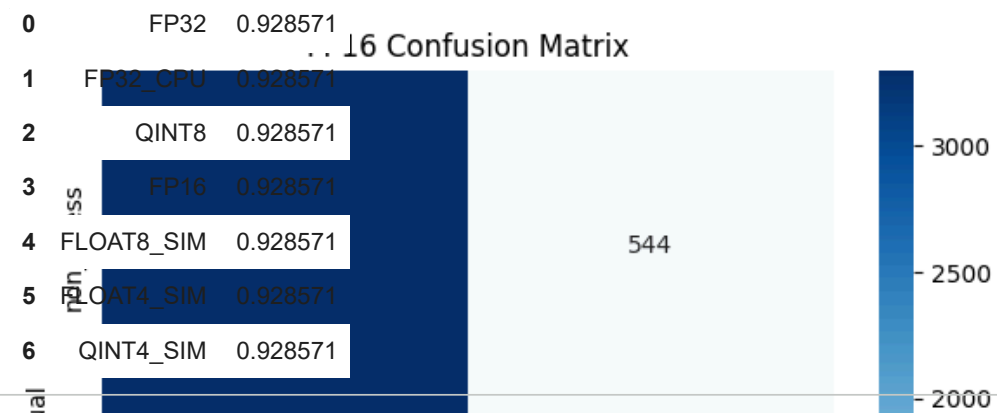
```

    "Accuracy": [
        acc_fp32,
        acc_fp32_cpu,
        acc_qint8,
        acc_fp16,
        acc_float8,
        acc_float4,
        acc_qint4
    ]
})

display(comparison)

```

Correct : 26
Wrong : 2 Model Accuracy



```
## Pilih quantization terbaik dari data asli kamu
```

```
print("="*50)
print("CUSTOM DATA RESULT")
print("="*50)

print(f"FP16 : {acc_fp16:.4f}")
print(f"QINT8 : {acc_qint8:.4f}")
```

```
if acc_fp16 >= acc_qint8:
    best_quant = "FP16"
    best_model = fp16_model
else:
    best_quant = "QINT8"
    best_model = qint8_model
```

```
print()
print("BEST QUANTIZATION :", best_quant)
```

```
=====
CUSTOM DATA RESULT
=====
FP16   : 0.9286
QINT8  : 0.9286

BEST QUANTIZATION : FP16
```

F. QUANTIZATION/PRECISION VARIANTS

```
fp32_model.load_state_dict(
    torch.load("best_emotion_model.pth")
)
```

```
<All keys matched successfully>
```

```
# F.1 UTILITAS UKURAN MODEL

def get_model_size_kb(model, path='temp_model.pth'):
    torch.save(model.state_dict(), path)
    size_kb = os.path.getsize(path) / 1024
    os.remove(path)
    return size_kb

def get_object_size_kb(obj, path='temp_obj.pth'):
    torch.save(obj, path)
    size_kb = os.path.getsize(path) / 1024
    os.remove(path)
    return size_kb
```

```
import shutil

# =====
# SAVE FINAL CNN (BEFORE QUANTIZATION)
# =====

shutil.copy(
    "best_emotion_model.pth",
    "emotion_cnn_fp32.pth"
)

print("CNN FP32 berhasil disimpan sebagai emotion_cnn_fp32.pth")
```

CNN FP32 berhasil disimpan sebagai emotion_cnn_fp32.pth

```
# F.2 FP32 – BASELINE
fp32_device = ( 'cuda'
    if torch.cuda.is_available()
    else 'cpu')

fp32_model = EmotionCNNDeepFC(num_classes=num_classes)

fp32_model.load_state_dict(
    torch.load('best_emotion_model.pth',map_location='cpu')
)

fp32_model.eval().to(fp32_device)
fp32_size = get_model_size_kb(fp32_model)
print(f'FP32 model size: {fp32_size:.2f} KB')
models_to_test = {}

# FP32 GPU
models_to_test['fp32'] = {
    'model': fp32_model,
    'device': fp32_device,
    'size_kb': fp32_size,
    'note': 'Baseline full precision on GPU'
}

# FP32 CPU
models_to_test['fp32_cpu'] = {
    'model': fp32_model.cpu(),
    'device': 'cpu',
    'size_kb': fp32_size,
    'note': 'Baseline full precision on CPU'
}
```

FP32 model size: 44094.61 KB

```
# F.3 QINT8 – Dynamic Quantization (torch.ao)

qint8_base_model = EmotionCNNDeepFC(
    num_classes=num_classes
)

qint8_base_model.load_state_dict(
    torch.load(
        'best_emotion_model.pth',
        map_location='cpu'
    )
)

qint8_base_model.eval()
qint8_model = torch.ao.quantization.quantize_dynamic(
    qint8_base_model,
    {nn.Linear},
    dtype=torch.qint8
)

qint8_model.eval()
models_to_test['qint8'] = {
    'model': qint8_model,

    'device': 'cpu',

    'size_kb': get_object_size_kb(qint8_model),

    'note': 'Dynamic INT8 quantization using torch.ao.quantization'
}
```

/tmp/ipykernel_1397/1383549148.py:15: DeprecationWarning: torch.ao.quantization is deprecated and will be removed in 2.10.
For migrations of users:
1. Eager mode quantization (torch.ao.quantization.quantize, torch.ao.quantization.quantize_dynamic), please migrate to use torchao eager mode
2. FX graph mode quantization (torch.ao.quantization.quantize_fx.prepare_fx,torch.ao.quantization.quantize_fx.convert_fx, please migrate to u
3. pt2e quantization has been migrated to torchao (<https://github.com/pytorch/ao/tree/main/torchao/quantization/pt2e>)
see <https://github.com/pytorch/ao/issues/2259> for more details
qint8_model = torch.ao.quantization.quantize_dynamic(

```
# =====
# SAVE QUANTIZED MODEL
```

```
# =====
```

```
torch.save(
    qint8_model.state_dict(),
    "emotion_cnn_qint8.pth"
)

print("Quantized model berhasil disimpan.")
```

```
Quantized model berhasil disimpan.
```

```
import os

for file in os.listdir():
    if file.endswith(".pth"):
        print(file)
```

```
best_emotion_model.pth
emotion_cnn_qint8.pth
emotion_cnn_fp32.pth
```

```
# F.4 FLOAT16 – Half Precision
# FP16 lebih efisien di GPU; di CPU bisa lambat untuk beberapa operasi
fp16_device = 'cuda' if torch.cuda.is_available() else 'cpu'
fp16_model = EmotionCNNDeepFC(num_classes=num_classes)
fp16_model.load_state_dict(torch.load('best_emotion_model.pth', map_location='cpu'))
fp16_model = fp16_model.half().eval().to(fp16_device)

models_to_test['float16'] = {
    'model': fp16_model,
    'device': fp16_device,
    'size_kb': fp32_size * 0.5, # estimasi: 16-bit = setengah 32-bit
    'note': 'Half precision (FP16). Optimal di GPU.'
}
```

```
# F.5 EXPERIMENTAL FP8
# EXPERIMENTAL FP8 TEST

try:
    fp8_model = EmotionCNNDeepFC(
        num_classes=num_classes
    )

    fp8_model.load_state_dict(
        torch.load(
            'best_emotion_model.pth',
            map_location='cpu'
        )
    )

    fp8_model = fp8_model.to('cuda')

    # convert parameter ke float8
    for param in fp8_model.parameters():
        param.data = param.data.to(torch.float8_e4m3fn)
    print("FP8 conversion success")

except Exception as e:
    print("FP8 conversion failed")
    print(e)
```

```
FP8 conversion success
```

```
# TEST FP8 INFERENCE
try:
    dummy = torch.randn(
        1, 1, 48, 48,
        device='cuda'
    ).to(torch.float8_e4m3fn)
    output = fp8_model(dummy)
    print("FP8 inference success")
    print("Output shape:", output.shape)

except Exception as e:
    print("FP8 inference failed")
    print(e)
```

```
FP8 inference failed
getCudnnDataTypeFromScalarType() not supported for Float8_e4m3fn
```

```
# Confution Matriks
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

def plot_quant_confusion_matrix(model, dataloader, device, title):

    model.eval()

    y_true = []
```

```

y_pred = []

with torch.no_grad():

    for images, labels in dataloader:

        images = images.to(device)
        labels = labels.to(device)

        # FP16
        if next(model.parameters()).dtype == torch.float16:
            images = images.half()

        outputs = model(images)

        preds = outputs.argmax(dim=1)

        y_true.extend(labels.cpu().numpy())
        y_pred.extend(preds.cpu().numpy())

cm = confusion_matrix(y_true, y_pred)

fig, ax = plt.subplots(figsize=(5,5))

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Non Stress","Stress"]
)

disp.plot(
    cmap="Blues",
    values_format="d",
    ax=ax
)

plt.title(title)

plt.show()

```

```

# F.5 SIMULATED LOW PRECISION VARIANTS
# FP8 / FP4 / QINT4 belum didukung native secara stabil
# pada PyTorch standar dan hardware umum.
#
# Oleh karena itu:
# - ukuran model disimulasikan berdasarkan rasio bit
# - accuracy dan latency masih menggunakan fallback FP32
# - digunakan untuk analisis potensi kompresi model

simulated_variants = {
    'float8_simulated': 8,
    'float4_simulated': 4,
    'qint4_simulated': 4,
}

for name, bits in simulated_variants.items():

    estimated_size = fp32_size * (bits / 32)

    models_to_test[name] = {
        'model': fp32_model,
        'device': 'cuda',
        'size_kb': estimated_size,
        'note': f'Simulated {bits}-bit - size estimation only'
    }

print('SIMULATED LOW PRECISION VARIANTS')

for k, v in models_to_test.items():
    print(f'''
Variant : {k}
Size KB : {v["size_kb"]:.2f}
Note    : {v["note"]}
''')

```

SIMULATED LOW PRECISION VARIANTS

Variant : fp32
Size KB : 44094.61
Note : Baseline full precision on GPU

Variant : fp32_cpu
Size KB : 44094.61
Note : Baseline full precision on CPU

Variant : qint8
Size KB : 14439.53
Note : Dynamic INT8 quantization using torch.ao.quantization

Variant : float16

Size KB : 22047.31
Note : Half precision (FP16). Optimal di GPU.

Variant : float8_simulated
Size KB : 11023.65
Note : Simulated 8-bit – size estimation only

Variant : float4_simulated
Size KB : 5511.83
Note : Simulated 4-bit – size estimation only

Variant : qint4_simulated
Size KB : 5511.83
Note : Simulated 4-bit – size estimation only

```
# F.6 ANALISIS LOW-BIT QUANTIZATION
print("LOW-BIT QUANTIZATION ANALYSIS")

print("""
Float8, Float4, dan QINT4 digunakan sebagai simulasi
low precision quantization untuk melihat potensi
pengurangan ukuran model pada deployment masa depan.

Semakin kecil precision:
- ukuran model semakin kecil
- kebutuhan memori semakin rendah
- potensi deployment embedded semakin baik

Namun:
- stabilitas numerik dapat menurun
- akurasi dapat berkurang
- hardware support masih terbatas
""")
```

LOW-BIT QUANTIZATION ANALYSIS

Float8, Float4, dan QINT4 digunakan sebagai simulasi low precision quantization untuk melihat potensi pengurangan ukuran model pada deployment masa depan.

Semakin kecil precision:

- ukuran model semakin kecil
- kebutuhan memori semakin rendah
- potensi deployment embedded semakin baik

Namun:

- stabilitas numerik dapat menurun
- akurasi dapat berkurang
- hardware support masih terbatas

```
# F.7 ESTIMASI KOMPRESI MODEL

baseline_size = fp32_size

print("ESTIMATED COMPRESSION RATIO")

for name in simulated_variants.keys():

    variant_size = models_to_test[name]['size_kb']

    compression_ratio = baseline_size / variant_size

    print(f'''
{name}

Original Size : {baseline_size:.2f} KB
Compressed   : {variant_size:.2f} KB
Compression  : {compression_ratio:.2f}x
''')
```

ESTIMATED COMPRESSION RATIO

float8_simulated

Original Size : 44094.61 KB
Compressed : 11023.65 KB
Compression : 4.00x

float4_simulated

Original Size : 44094.61 KB
Compressed : 5511.83 KB
Compression : 8.00x

qint4_simulated

Original Size : 44094.61 KB
Compressed : 5511.83 KB

Compression : 8.00x

```
# ESTIMASI KOMPRESI MODEL
compression_labels = []
compression_values = []

for name in simulated_variants.keys():
    variant_size = models_to_test[name]['size_kb']

    ratio = baseline_size / variant_size

    compression_labels.append(name)
    compression_values.append(ratio)

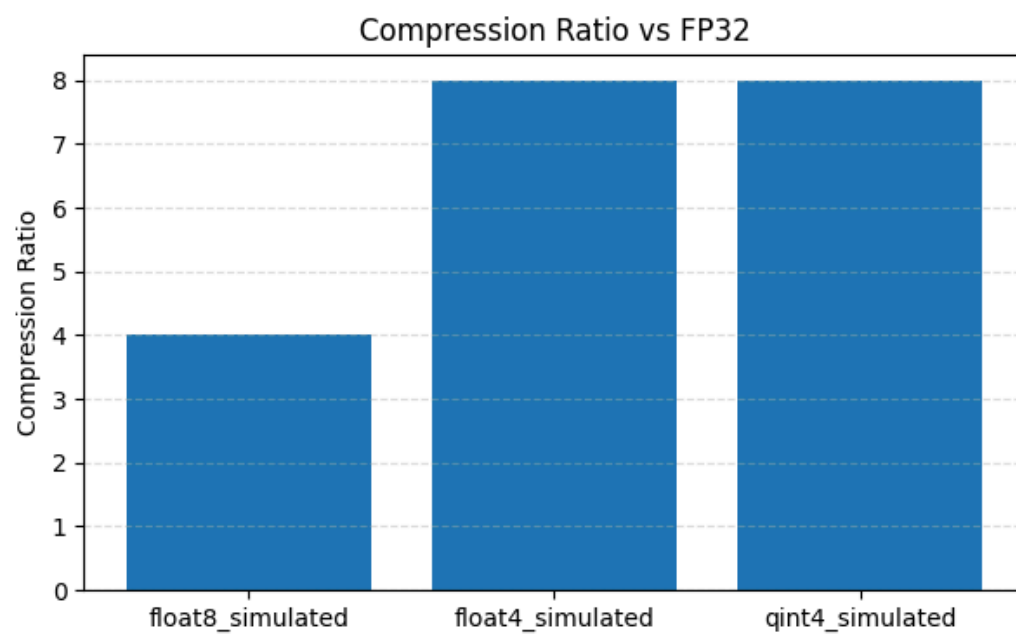
plt.figure(figsize=(7,4))

plt.bar(compression_labels, compression_values)

plt.ylabel("Compression Ratio")
plt.title("Compression Ratio vs FP32")

plt.grid(axis='y', linestyle='--', alpha=0.4)

plt.show()
```



```
# F.8 VISUALISASI LOW-BIT MODEL SIZE

lowbit_names = list(simulated_variants.keys())

lowbit_sizes = [
    models_to_test[k]['size_kb']
    for k in lowbit_names
]

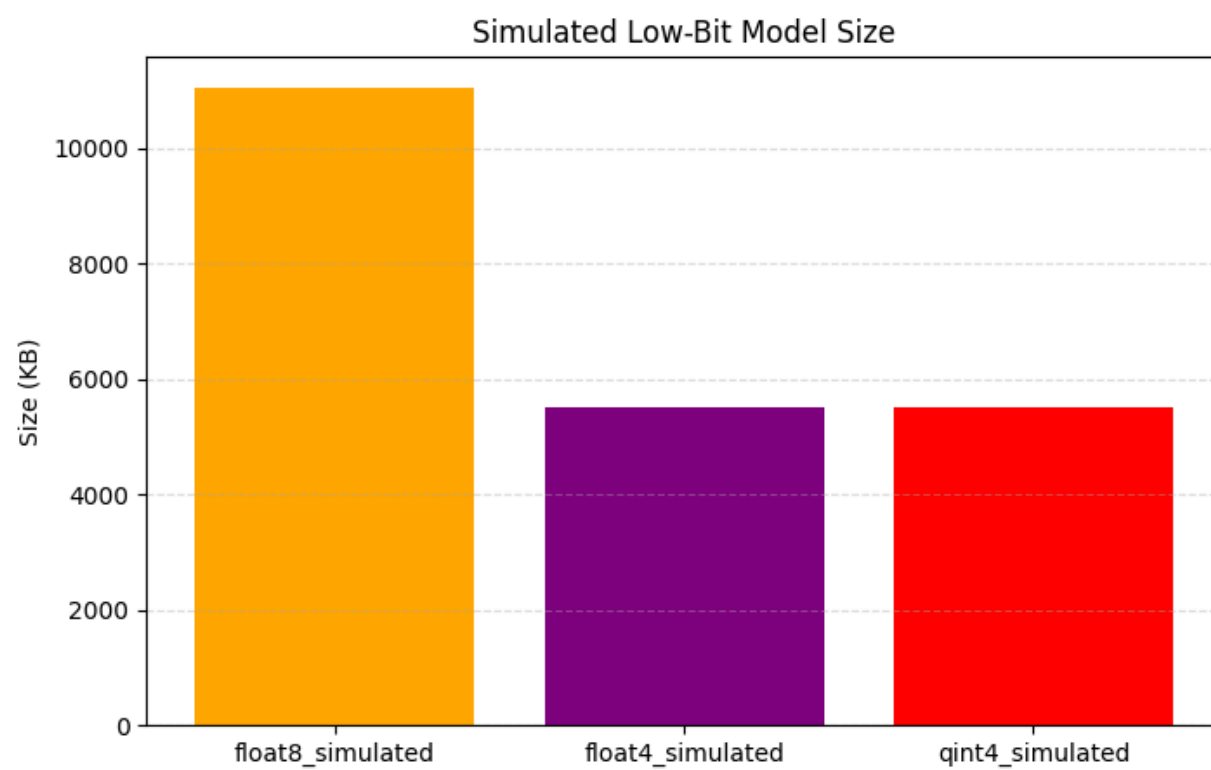
plt.figure(figsize=(8,5))

plt.bar(
    lowbit_names,
    lowbit_sizes,
    color=['orange', 'purple', 'red']
)

plt.title('Simulated Low-Bit Model Size')
plt.ylabel('Size (KB)')

plt.grid(axis='y', linestyle='--', alpha=0.4)

plt.show()
```



```
# F.9 POTENSI DEPLOYMENT LOW-BIT

deployment_notes = {
    'float8_simulated':
        'Potensi deployment lightweight GPU inference.',

    'float4_simulated':
        'Potensi ultra lightweight embedded AI.',

    'qint4_simulated':
        'Potensi deployment edge AI dan IoT device.'
}

print("LOW-BIT DEPLOYMENT POTENTIAL")

for k, v in deployment_notes.items():

    print(f'''
{k}
→ {v}
''')
```

```
LOW-BIT DEPLOYMENT POTENTIAL

float8_simulated
→ Potensi deployment lightweight GPU inference.

float4_simulated
→ Potensi ultra lightweight embedded AI.

qint4_simulated
→ Potensi deployment edge AI dan IoT device.
```

G. COMPARISON - ACCURACY, SIZE, LATENCY

```
# G.1 EVALUASI AKURASI SEMUA VARIANT
@torch.no_grad()
def evaluate_variant(model, loader, variant_device='cpu', use_half=False):
    model.eval()
    model = model.to(variant_device)
    correct, total = 0, 0
    for images, labels in loader:
        images = images.to(variant_device)
        labels = labels.to(variant_device)
        if use_half:
            images = images.half()
        outputs = model(images)
        _, preds = outputs.max(1)
        correct += preds.eq(labels).sum().item()
        total += labels.size(0)
    return correct / total

accuracy_results = {}
for name, item in models_to_test.items():
    use_half = (name == 'float16')
    try:
        acc = evaluate_variant(item['model'], val_loader,
                               variant_device=item['device'], use_half=use_half)
    except Exception as e:
        print(f'{name} error saat evaluasi: {e}')
        acc = None
```



```
accuracy_results[name] = acc
print(f'{name:<22} accuracy: {acc}')
```

fp32	accuracy: 0.8497028021511464
fp32_cpu	accuracy: 0.8497028021511464
qint8	accuracy: 0.8494197565808095
float16	accuracy: 0.8497028021511464
float8_simulated	accuracy: 0.8497028021511464
float4_simulated	accuracy: 0.8497028021511464
qint4_simulated	accuracy: 0.8497028021511464

```
# G.2 LATENCY / KECEPATAN INFERENSI
@torch.no_grad()
def measure_latency(model, variant_device='cuda', use_half=False,
                    n_warmup=20, n_runs=100):
    model.eval()
    model = model.to(variant_device)
    dummy = torch.randn(1, 1, 48, 48).to(variant_device)
    if use_half:
        dummy = dummy.half()
    # Warmup
    for _ in range(n_warmup):
        _ = model(dummy)
    if variant_device == 'cuda':
        torch.cuda.synchronize()
    # Timing
    start = time.perf_counter()
    for _ in range(n_runs):
        _ = model(dummy)
    if variant_device == 'cuda':
        torch.cuda.synchronize()
    end = time.perf_counter()
    return ((end - start) / n_runs) * 1000    # ms

latency_results = {}
for name, item in models_to_test.items():
    use_half = (name == 'float16')
    try:
        lat = measure_latency(item['model'], variant_device=item['device'],
                              use_half=use_half)
    except Exception as e:
        print(f'{name} error saat latency test: {e}')
        lat = None
    latency_results[name] = lat
    print(f'{name:<22} latency: {lat:.4f} ms' if lat else f'{name} latency: None')
```

fp32	latency: 1.7496 ms
fp32_cpu	latency: 19.9335 ms
qint8	latency: 11.8316 ms
float16	latency: 1.4071 ms
float8_simulated	latency: 1.2466 ms
float4_simulated	latency: 1.2542 ms
qint4_simulated	latency: 1.5154 ms

```
print("""
Catatan:
Latency antar varian tidak sepenuhnya apple-to-apple
karena FP32 dan FP16 diuji pada GPU,
sedangkan QINT8 diuji pada CPU.

Oleh karena itu latency digunakan
sebagai indikasi deployment scenario,
bukan perbandingan hardware identik.
""")
```

Catatan:

Latency antar varian tidak sepenuhnya apple-to-apple karena FP32 dan FP16 diuji pada GPU, sedangkan QINT8 diuji pada CPU.

Oleh karena itu latency digunakan sebagai indikasi deployment scenario, bukan perbandingan hardware identik.

```
# G.3 TABEL PERBANDINGAN FINAL
baseline_acc = accuracy_results['fp32']
rows = []
for name, item in models_to_test.items():
    acc = accuracy_results[name]
    latency = latency_results[name]
    size = item['size_kb']

    acc_drop = (
        baseline_acc - acc
        if acc is not None and baseline_acc is not None
        else None
    )

    rows.append({
        'Model Variant': name,
```

```

        'Type': (
            'Simulated'
            if 'simulated' in name
            else 'Real'
        ),
        'Size (KB)': round(size, 2),
        'Val Accuracy': (
            round(acc, 4)
            if acc is not None
            else None
        ),
        'Accuracy Drop vs FP32': (
            round(acc_drop, 6)
            if acc_drop is not None
            else None
        ),
        'Latency ms/image': (
            round(latency, 4)
            if latency is not None
            else None
        ),
        'Device': item['device'],
        'Note': item['note']
    })

result_df = pd.DataFrame(rows)

# Pisahkan real vs simulated
real_df = result_df[
    result_df['Type'] == 'Real'
].copy()
sim_df = result_df[
    result_df['Type'] == 'Simulated'
].copy()

# Sorting
real_df = real_df.sort_values(
    by=['Val Accuracy', 'Latency ms/image'],
    ascending=[False, True]
).reset_index(drop=True)
sim_df = sim_df.sort_values(
    by=['Size (KB)'],
    ascending=[True]
).reset_index(drop=True)

# Gabungkan kembali
result_df = pd.concat(
    [real_df, sim_df],
    ignore_index=True
)

print("=" * 90)
print("FINAL MODEL COMPARISON")
print("=" * 90)

print(result_df.to_string(index=False))

print("""
Catatan:
- FP32 dan FP16 diuji pada GPU
- QINT8 diuji pada CPU
- FLOAT8, FLOAT4, dan QINT4 masih simulasi
- Latency digunakan sebagai indikasi deployment
""")

result_df

```

=====

FINAL MODEL COMPARISON

=====

Model Variant	Type	Size (KB)	Val Accuracy	Accuracy Drop vs FP32	Latency ms/image	Device	
float16	Real	22047.31	0.8497	0.000000	1.4071	cuda	Half precision (FP16). Opt
fp32	Real	44094.61	0.8497	0.000000	1.7496	cuda	Baseline full prec
fp32_cpu	Real	44094.61	0.8497	0.000000	19.9335	cpu	Baseline full prec
qint8	Real	14439.53	0.8494	0.000283	11.8316	cpu	Dynamic INT8 quantization using torch.ao.
float4_simulated	Simulated	5511.83	0.8497	0.000000	1.2542	cuda	Simulated 4-bit – size est
qint4_simulated	Simulated	5511.83	0.8497	0.000000	1.5154	cuda	Simulated 4-bit – size est
float8_simulated	Simulated	11023.65	0.8497	0.000000	1.2466	cuda	Simulated 8-bit – size est

Catatan:

- FP32 dan FP16 diuji pada GPU
- QINT8 diuji pada CPU
- FLOAT8, FLOAT4, dan QINT4 masih simulasi
- Latency digunakan sebagai indikasi deployment

	Model Variant	Type	Size (KB)	Val Accuracy	Accuracy Drop vs FP32	Latency ms/image	Device	Note
0	float16	Real	22047.31	0.8497	0.000000	1.4071	cuda	Half precision (FP16). Optimal di GPU.
1	fp32	Real	44094.61	0.8497	0.000000	1.7496	cuda	Baseline full precision on GPU
2	fp32_cpu	Real	44094.61	0.8497	0.000000	19.9335	cpu	Baseline full precision on CPU
3	qint8	Real	14439.53	0.8494	0.000283	11.8316	cpu	Dynamic INT8 quantization using torch.ao.quant...
4	float4_simulated	Simulated	5511.83	0.8497	0.000000	1.2542	cuda	Simulated 4-bit — size estimation only
5	qint4_simulated	Simulated	5511.83	0.8497	0.000000	1.5154	cuda	Simulated 4-bit — size estimation only

G.4 VISUALISASI PERBANDINGAN

```
variants = result_df['Model Variant'].tolist()
sizes = result_df['Size (KB)'].tolist()
accuracies= result_df['Val Accuracy'].tolist()
latencies = result_df['Latency ms/image'].tolist()

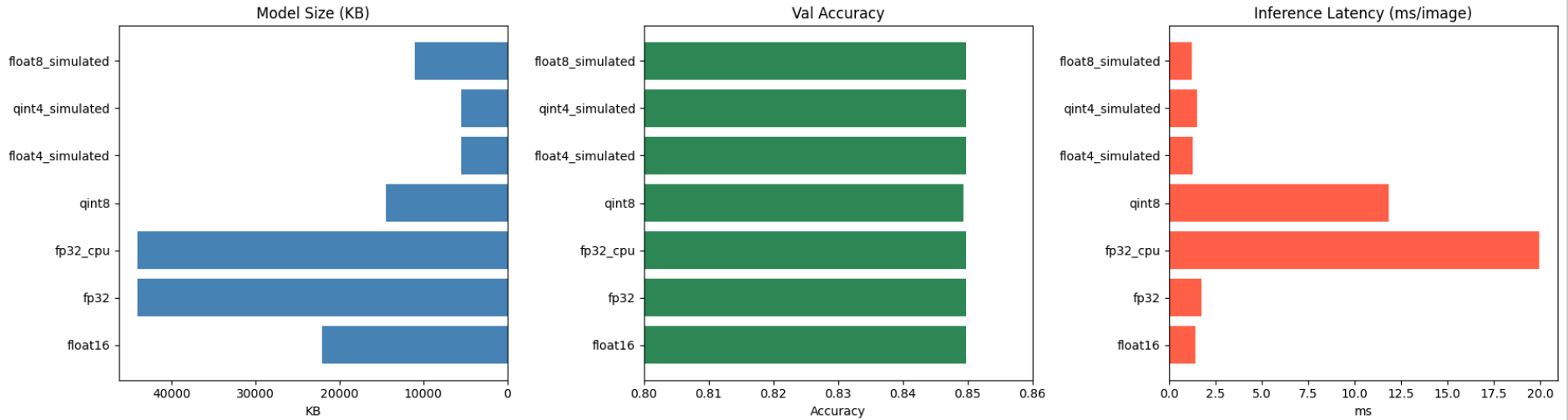
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

axes[0].barh(variants, sizes, color='steelblue')
axes[0].set_title('Model Size (KB)'); axes[0].set_xlabel('KB')
axes[0].invert_xaxis()

axes[1].barh(variants, accuracies, color='seagreen')
axes[1].set_title('Val Accuracy'); axes[1].set_xlabel('Accuracy')
axes[1].set_xlim(0.80, 0.86)

axes[2].barh(variants, latencies, color='tomato')
axes[2].set_title('Inference Latency (ms/image)'); axes[2].set_xlabel('ms')

plt.tight_layout(); plt.show()
```



```
# G.5 CLASSIFICATION REPORT
@torch.no_grad()
def get_predictions(model, loader, eval_device=device):
    model.eval()
    all_preds, all_labels = [], []
    for images, labels in loader:
        images = images.to(eval_device)
        outputs = model(images)
        _, preds = outputs.max(1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.numpy())
    return np.array(all_labels), np.array(all_preds)

y_true, y_pred = get_predictions(model, val_loader)
class_names = train_dataset.classes

report = classification_report(y_true, y_pred, target_names=class_names, output_dict=True)
```

```
# PLOT CLASSIFICATION REPORT

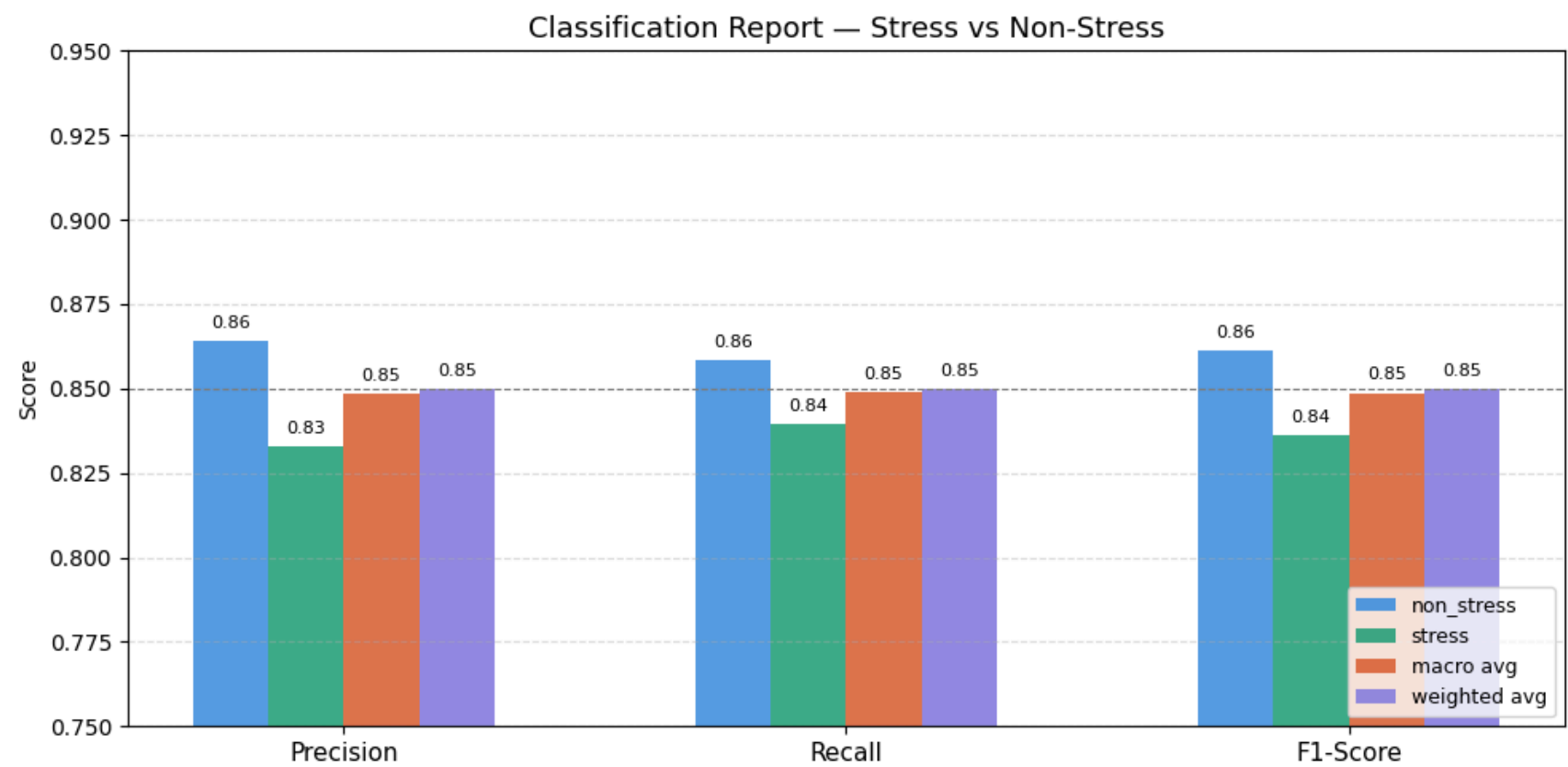
metrics = ['precision', 'recall', 'f1-score']
categories = class_names + ['macro avg', 'weighted avg']
x = np.arange(len(metrics))
width = 0.15
colors = ['#378ADD', '#1D9E75', '#D85A30', '#7F77DD', '#BA7517']

fig, ax = plt.subplots(figsize=(10, 5))

for i, cat in enumerate(categories):
    vals = [report[cat][m] for m in metrics]
    bars = ax.bar(x + i * width, vals, width, label=cat, color=colors[i], alpha=0.85)
    for bar, val in zip(bars, vals):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.003,
                f'{val:.2f}', ha='center', va='bottom', fontsize=8)

ax.set_ylim(0.75, 0.95)
ax.set_xticks(x + width * (len(categories) - 1) / 2)
ax.set_xticklabels(['Precision', 'Recall', 'F1-Score'], fontsize=11)
ax.set_ylabel('Score')
ax.set_title('Classification Report – Stress vs Non-Stress', fontsize=13)
ax.legend(loc='lower right', fontsize=9)
ax.axhline(y=report['accuracy'], color='gray', linestyle='--', linewidth=0.8, label='Overall Accuracy')
ax.grid(axis='y', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()

# Tetap print tabel aslinya juga
print("\n Classification Report")
print(classification_report(y_true, y_pred, target_names=class_names))
print(f'\nOverall Accuracy: {report["accuracy"]:.4f}')
```



Classification Report				
	precision	recall	f1-score	support
non_stress	0.86	0.86	0.86	3838
stress	0.83	0.84	0.84	3228
accuracy			0.85	7066
macro avg	0.85	0.85	0.85	7066
weighted avg	0.85	0.85	0.85	7066

Overall Accuracy: 0.8497

```
# G.6 ROC CURVE dan AUC
from sklearn.metrics import roc_curve, auc, roc_auc_score
@torch.no_grad()
def get_probs(model, loader, eval_device=device):
    model.eval()
    all_probs, all_labels = [], []
    for images, labels in loader:
        images = images.to(eval_device)
        outputs = model(images)
        probs = torch.softmax(outputs, dim=1)
        # Probabilitas kelas stress
        stress_probs = probs[:, 1].cpu().numpy()
        all_probs.extend(stress_probs)
        all_labels.extend(labels.numpy())
    return np.array(all_labels), np.array(all_probs)
y_true_auc, y_score = get_probs(model, val_loader)
roc_auc = roc_auc_score(y_true_auc, y_score)
fpr, tpr, thresholds = roc_curve(y_true_auc, y_score)
```

```

plt.figure(figsize=(7,6))

plt.plot(
    fpr,
    tpr,
    linewidth=3,
    label=f'ROC Curve (AUC = {roc_auc:.4f})'
)

plt.plot([0,1],[0,1], linestyle='--')

plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)

plt.title(
    f'ROC Curve – Stress vs Non-Stress\nAUC = {roc_auc:.4f}',
    fontsize=15,
    fontweight='bold'
)

plt.legend(fontsize=12)

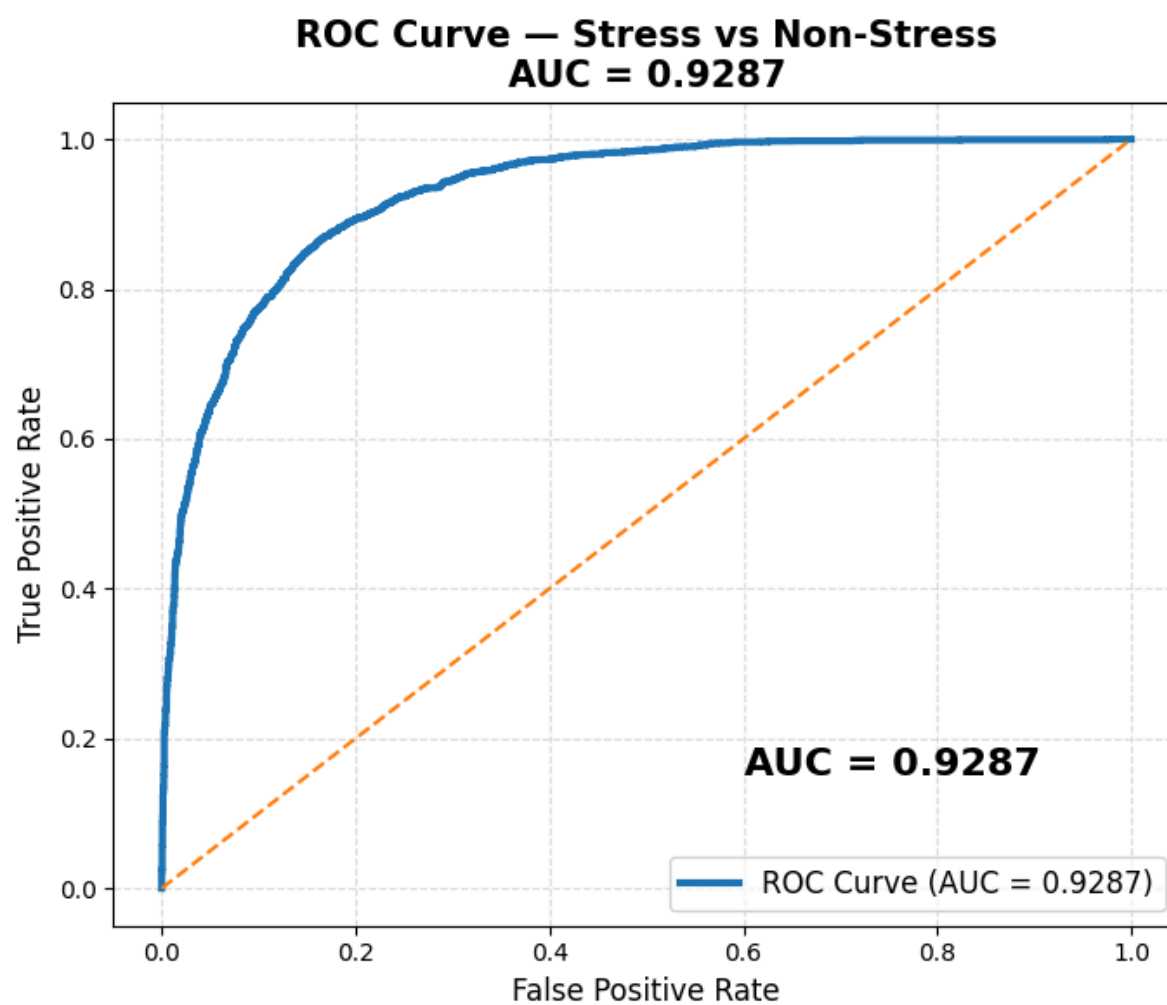
plt.text(
    0.60,
    0.15,
    f'AUC = {roc_auc:.4f}',
    fontsize=16,
    fontweight='bold'
)

plt.grid(True, linestyle='--', alpha=0.4)

plt.tight_layout()
plt.show()

print(f'ROC-AUC Score: {roc_auc:.4f}')
print(f'ROC-AUC Score: {roc_auc:.4f}')

```



```

ROC-AUC Score: 0.9287
ROC-AUC Score: 0.9287

```

```

# G.6 ROC CURVE + AUC SCORE + DIAGONAL BASELINE
from sklearn.metrics import roc_curve, roc_auc_score, auc

@torch.no_grad()
def get_prediction_scores(model, loader, eval_device=device):
    model.eval()

    all_labels = []
    all_scores = []

    for images, labels in loader:
        images = images.to(eval_device)

        outputs = model(images)
        probs = torch.softmax(outputs, dim=1)

        # Ambil probabilitas kelas positif: stress
        stress_scores = probs[:, 1].detach().cpu().numpy()

```

```
        all_scores.extend(stress_scores)
        all_labels.extend(labels.numpy())

    return np.array(all_labels), np.array(all_scores)

# Ambil label asli dan probability score
y_true_auc, y_score = get_prediction_scores(model, val_loader, device)

# Hitung ROC dan AUC
fpr, tpr, thresholds = roc_curve(y_true_auc, y_score)
roc_auc = roc_auc_score(y_true_auc, y_score)

nprint(f"ROC-AUC Score: {roc_auc:.4f}")
```

ROC-AUC Score: 0.9287